



ICE: An Intelligent Cognition Engine with 3D NAND-based In-Memory Computing for Vector Similarity Search Acceleration

Han-Wen Hu^{*†}, Wei-Chen Wang^{‡*§}, Yuan-Hao Chang[¶], Yung-Chun Lee^{*}, Bo-Rong Lin^{*}, Huai-Mu Wang^{*}, Yen-Po Lin^{*||}, Yu-Ming Huang^{*}, Chong-Ying Lee^{*}, Tzu-Hsiang Su^{*}, Chih-Chang Hsieh^{*}, Chia-Ming Hu^{*}, Yi-Ting Lai^{*}, Chung-Kuang Chen^{*}, Han-Sung Chen^{*}, Hsiang-Pang Li^{*}, Tei-Wei Kuo^{§||††**}, Meng-Fan Chang[†], Keh-Chung Wang^{*}, Chun-Hsiung Hung^{*}, and Chih-Yuan Lu^{*}

^{*}Macronix International Co., Ltd., Email:paddingtonhu@mxic.com.tw

[†]Department of Electrical Engineering, National Tsing Hua University, Email:s109061802@m109.nthu.edu.tw

[‡]Department of Electrical Engineering and Computer Science, Massachusetts Institute of Technology

[§]Department of Computer Science and Information Engineering, National Taiwan University

[¶]Institute of Information Science, Academia Sinica

^{||}Graduate Institute of Electronics Engineering, National Taiwan University

^{††}Graduate Institute of Networking and Multimedia, National Taiwan University

^{**}High Performance and Scientific Computing Center, National Taiwan University

Abstract—Vector similarity search (VSS) for unstructured vectors generated via machine learning methods is a promising solution for many applications, such as face search. With increasing awareness and concern about data security requirements, there is a compelling need to store data and process VSS applications locally on edge devices rather than send data to servers for computation. However, the explosive amount of data movement from NAND storage to DRAM across memory hierarchy and data processing of the entire dataset consume enormous energy and require long latency for VSS applications. Specifically, edge devices with insufficient DRAM capacity will trigger data swap and deteriorate the execution performance. To overcome this crucial hurdle, we propose an *intelligent cognition engine (ICE)* with *cognitive 3D NAND*, featuring *non-volatile in-memory computing (nvIMC)* to accelerate the processing, suppress the data movement, and reduce data swap between the processor and storage. This cognitive 3D NAND features *digital nvIMC techniques (i.e., ADC/DAC-free approach)*, *high-density 3D NAND*, and compatibility with standard 3D NAND products with minor modifications. To facilitate parallel INT8/INT4 vector-vector multiplication (VVM) and mitigate the reliability issue of 3D NAND, we develop a *bit-error-tolerance data encoding* and a *two's complement-based digital accumulator*. VVM can support similarity computations (e.g., cosine similarity and Euclidean distance), which are required to search “the most similar data” right where they are stored. In addition, the proposed solution can be realized on edge storage products, e.g., embedded Multi-Media Card (eMMC). The measured and simulated results on real 3D NAND chips show that ICE enhances the system execution time by 17× to 95× and energy efficiency by 11× to 140×, compared to traditional von Neumann approaches using state-of-the-art edge systems with MobileFaceNet on CASIA-WebFace dataset. To the best of our knowledge, this work demonstrates the first 3D NAND-based digital nvIMC technique with measured silicon data.

Keywords-3D NAND, In-Memory Computing, Vector Similarity Search, Unstructured Data Search

I. INTRODUCTION

In recent years, artificial intelligence (AI) applications have made remarkable achievements and successes in various fields, such as image retrieval [77], [88], audio texturing [62], [71], etc. To achieve much higher accuracy for each device/user, customization by training/fine-tuning datasets based on local data [93] is a widely-adopted approach. However, training/fine-tuning local datasets consumes significant human resources and time at a finite cost in practice. To address these issues, vector similarity search (VSS) [66], [69] is rapidly gaining ground because of its wide variety of applications with a large customized dataset.

The workflow of general VSS applications is primarily composed of two components/processes, a feature extractor, and a classifier. The feature extractor remaps feature representations from customized raw data to an extracted feature vector dataset by extracting unstructured feature vectors via machine learning methods and representing the objects in high-dimensional space. For instance, each human face can be extracted as a 512-dimensional feature vector where each dimension represents a feature of the face, such as hair color or lip shape. The classifier then utilizes the extracted feature vector dataset to conduct the similarity search (e.g., cosine similarity [32], [67], Euclidean distance [34], [57], and Hamming distance [68], [101]) by comparing the similarity scores between the input query/vector and the entire extracted feature vector dataset. By this means, VSS is able to avoid the significant training/fine-tuning overheads in conventional AI applications.

There are a wide variety of VSS applications in our daily lives. For example, face recognition, which can be utilized for smart retail and access control, is one of the hot topics of VSS

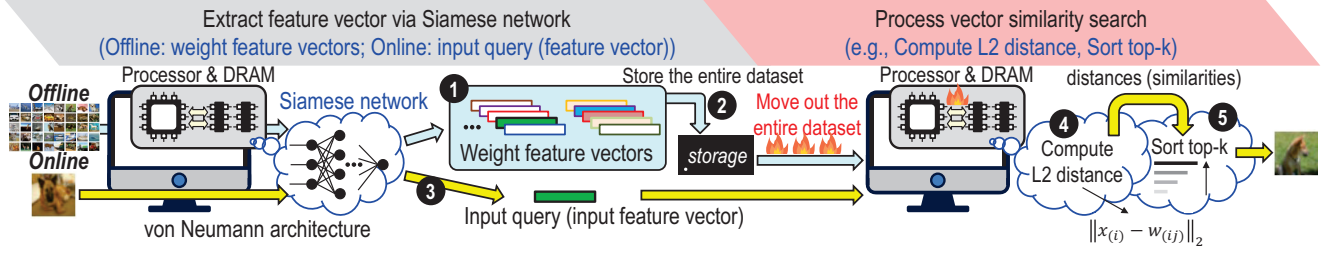


Figure 1: The general workflow of VSS system with Siamese feature representation network.

applications in the commercial market [3], [4]. Visual search or content-based image retrieval, which can be exploited in E-commercial systems, is also a popular research topic in recent years because of the explosive amount of online images on social media and search engines [48], [102]. In fact, VSS applications are widely used and explored in both industry [8], [51], [98] and academia [84].

However, the massive amount of data movement of the entire extracted feature vector dataset from storage to processor across memory hierarchy and dataset processing consume high search energy (E_{SR}) and require a long search latency (t_{SR}) for VSS applications from a system perspective. Besides, the ability to conduct VSS applications and store the relevant data on local edge devices, rather than sending them to servers for computation, is in urgent demand owing to the rising awareness of data security [20], [56], [70], [90]. Motivated by this, this work aims to enable similarity search for VSS applications over edge devices by minimizing data movement, search latency, and energy.

Unfortunately, the solution of similarity search for VSS applications over edge devices is not straightforward. The capacity-hungry requirement in VSS applications makes it much harder to satisfy the memory density in von Neumann architectures, e.g., SRAM and DRAM, not to mention on resource-constrained edge devices. Insufficient memory density will trigger data swap and lead to large E_{SR} and long t_{SR} between the processor and storage. An intuitive solution is to employ multiple edge devices and partition the entire dataset into several small sub-datasets to compute the similarity search in parallel. However, this solution could significantly increase the cost. Meanwhile, employing a powerful general-purpose processor is overkill for such computing tasks [42], [52]. To reduce the overhead of excessive data movement, researchers proposed in-storage-computing for intelligent applications [25], [33], [66], [91]. They feature the near-data processing approaches that work by leveraging the controller's computing ability in the storage system to process the data right in the storage. However, most in-storage computing systems are still constructed based on the von Neumann architecture. The excessive data movement between the storage and controller still adversely affects the system performance [66].

To address high E_{SR} and long t_{SR} incurred by excessive data

movement, the nonvolatile in-memory computing (nvIMC) technique over NAND chips becomes a promising solution to realize high-dimensional *vector-vector multiplication* (VVM) for similarity search right where the data are stored. Moreover, high-density 3D NAND chips can support the capacity-hungry requirements for large feature vector datasets. Therefore, the 3D NAND-based nvIMC technique becomes a feasible solution for edge devices [17], [65], [92], suppressing the data movement or swap and offloading the computational burden of the processors. However, previous 3D NAND-based nvIMC designs [17], [65], [92] were mixed-signal works that introduced costly analog-to-digital converter and digital-to-analog converter (ADC/DAC) overheads, even without considering practical design issues. Moreover, these works merely presented preliminary ideas without evaluating the impact of bit error over accuracy.

This paper presents an *intelligent cognition engine* (ICE) with high-density digital-based nvIMC 3D NAND (cognitive 3D NAND) for VSS acceleration over edge devices. The overall performance of VSS applications can be enhanced by ICE with cognitive 3D NAND by leveraging the parallel processing power of multiple 3D NAND chips. We also fabricate a test chip for ICE to verify its feasibility and capability. To the best of our knowledge, this work is the first 3D NAND-based digital nvIMC design with real silicon data. Note that our methodology can support not only VVM operations but also support various computations.

The key contributions of this paper are listed as follows.

- 1) We propose *ICE with a high-density digital-based nvIMC 3D NAND* for VSS application acceleration. The proposed ICE can suppress data movement and swap, and decrease high E_{SR} and long t_{SR} on edge devices by adopting the digital nvIMC technique.
- 2) We design an *ADC/DAC-free* digital-based nvIMC technique featuring high density. Our proposed technique is compatible with standard 3D NAND products, which makes it more cost-effective.
- 3) We propose a *bit-error-tolerance (BET) data encoding* to mitigate the reliability issue and facilitate parallel VVM operations of 3D NAND by skipping the general error-correction-code decoding stage. To enable INT8/INT4 VVM, we propose a *two's complement-based (TC) digital accumulator* to overcome sign-bit

computations for VVM.

- 4) We conduct a series of experimental measurements and simulations to evaluate the feasibility and capability of ICE on real 3D NAND chips. ICE only requires 1.6% area overhead in commercial 3D NAND chips. Compared to the state-of-the-art edge systems, ICE improves execution time by $17\times$ to $95\times$ and energy efficiency by $11\times$ to $140\times$.

II. BACKGROUND AND MOTIVATION

A. Vector Similarity Search System Study

1) *Background on VSS*: General VSS systems use the Siamese feature representation network architecture [22], [29]. During training, the network learns to output similarity scores by calculating the similarity between an input query and feature vectors of a dataset. After training procedures, the model can be deployed to perform VSS applications in two phases, i.e., offline and online. Fig. 1 depicts the general VSS systems with Siamese feature representation network. In the offline phase, each image or data of a dataset will generate a feature vector via machine learning methods (①), and then the feature vector will be stored in the storage (②). Through the AI models, the system may obtain an input query in the online phase, i.e., a feature vector (③). After that, the VSS system scores the similarity between the given input query and every feature vector in the feature vector dataset via similarity computation (④). Eventually, the system sorts the scores to find top-k patterns (⑤), which match the given input query the most and then outputs relative contents (e.g., specific information) according to the top-k results.

Given the input query $x \in \mathbb{R}^d$ and the feature vector dataset $[w_i]_{i=0:n}$ ($w_i \in \mathbb{R}^d$), the top-k similarity search result L of x in terms of L_2 distance can be expressed as:

$$L = k\text{-argmin}_{i=0:n} \|x - w_i\|_2 \quad (1)$$

where d denotes the dimension of vector, and n denotes the number of feature vector. In practice, we usually decompose the last term in Equation 1 as follows:

$$\|x - w_i\|_2^2 = \|x\|^2 + \|w_i\|^2 - 2(x \cdot w_i) \quad (2)$$

The first two terms can be pre-calculated in one pass over the vector x and matrix w [41]. That is, *the bottleneck of VSS is to calculate the last term, i.e., $(x \cdot w_i)$, which is equivalent to the vector-matrix multiplication*. Meanwhile, the vector-matrix multiplication can be decomposed into multiple *vector-vector multiplications (VVM)*.

2) *VSS System Study and Observation*: The previous input-query acceleration designs [30] targeting face search applications [21] proposed to exhaustively search the entire feature vector dataset for similar results since the extracted input queries were unstructured [26], [27], [78]. While feature vector datasets are too large to be entirely stored in the main memory, existing works might need to iteratively load and search each part of the feature vector dataset from storage

(e.g., NAND flash memory) to the main memory. Instead of the similarity search time, the data movement time may dominate the entire system performance for VSS applications due to the excessive data movement between storage units and computing units. In addition, it is not cost-efficient to handle the similarity search by employing powerful processors. To verify the assumption, we conducted a preliminary experiment to implement the concept of existing VSS work [30] into three edge devices, namely Raspberry Pi3 [9], Raspberry Pi4 [10], and NVIDIA Jetson Nano [5].

Experiment Setup: We leverage the open-source validation set of CelebA [64], VGGFace2 [21] and CASIA-WebFace [100]. CelebA, VGGFace2, and CASIA-WebFace are about 20K, 170K, and 500K. Besides, as a case study of a VSS system, we implemented two AI models [26], [78] trained with triplet loss over Pytorch [72] and Tensorflow frameworks [11]. Note that the primary goal of the AI models trained with triplet loss is to minimize the distance between the “same category” of data points, maximize the distance between “different ones,” and eventually generate feature vectors [22], [26], [78].

Result Observation: Fig. 2 demonstrates the overall system performance results of adopting three edge devices with two AI models and three datasets for face search applications [26], [78]. We focus on the online phase instead of the offline phase since the online phase is much more critical in run time for VSS applications. We then break down the whole system execution time into two parts: the feature extraction latency (t_{FE}) and data search latency (t_{SR}). We observe that all the edge devices perform well over the small datasets (e.g., 20K faces), where their total execution time is usually short. On the contrary, with large datasets (e.g., 500K faces), the total execution time becomes relatively long and mainly dominated by t_{SR} , ranging from 90%-99%. In the meantime, E_{SR} also dominates the overall energy consumption. This rationale is mainly because of the insufficient memory (i.e., DRAM) density in edge devices for large datasets. Specifically, with a large amount of data, the system would continually trigger the memory swap, which swaps non-critical data from DRAM to internal/external storage to acquire free space in DRAM. Therefore, we observed that the similarity search becomes the primary performance bottleneck over large datasets in VSS applications. Furthermore, we also found that compared to general edge devices (e.g., Raspberry Pi3 and Pi4), t_{SR} and E_{SR} % of the total execution time of high-performance edge devices (e.g., NVIDIA Jetson Nano) is relatively higher due to less DRAM capacity. In summary, due to the insufficient memory issue and memory swap policy, the performance of edge devices could critically deteriorate in VSS applications.

B. 3D NAND-based nvIMC Study

This section introduces some background knowledge of 3D NAND features related to the operations in 3D NAND-

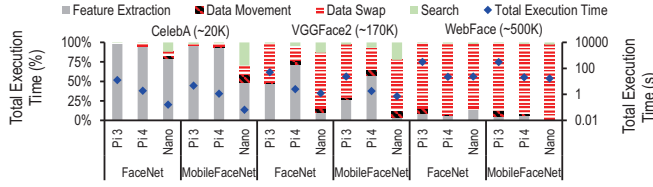


Figure 2: Measured system performance for implementing traditional face search on various edge devices. The searching time with large datasets (e.g., CASIA-WebFace, a 500K-face dataset) will dominate the total execution time on edge devices.

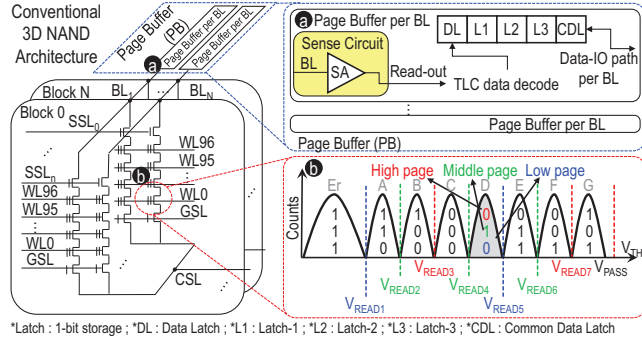


Figure 3: The conventional architecture of 3D NAND flash memory with (a) brief page buffer schematic and (b) conventional TLC 3D NAND cells.

based nvIMC. Then, we present the 3D NAND-based nvIMC designs proposed in previous works.

1) *3D NAND Flash Memory Introduction:* Generally, 3D NAND flash memory stores the target data into flash cells by performing the program and verifying operations and discriminates the target data stored in the selected cells by performing the read operation. For the read operation, reading the state of a NAND flash cell means discriminating one V_t level against another and disregarding V_t levels of all other cells in the string. The strings connect to the page buffer (PB) via bit-line (BL). Conventionally, PB per BL comprises a sensing circuit (SA), multiple latches, and logic components for multi-level sensing [45], [81], as shown in Fig. 3 (a). The word lines (WLs) in the selected block may rise to V_{READ} in the sensing operation. Afterward, other unselected cells are forced to use a pass voltage (V_{PASS}) at their WLs to act as a pass transistor. The V_{PASS} biased at WLs is larger than the maximum V_t level in general. At the end of the sense, the sense amplifier (SA) would discriminate the data according to the voltage level higher or lower than its SA threshold and store it in the data latch (DL). As shown in Fig. 3 (b), triple-level-cell (TLC) NAND cells apply seven V_{READ} to discriminate the low, middle, and high data. For instance, to discriminate the middle page data, TLC NAND cells need to apply three read operations (i.e., V_{READ2} , V_{READ4} , and V_{READ6}) in sequence. At the end of the multiple sensing operations, PB decodes the accurate data and stores it in DL. PB senses a NAND string of each BL to increase throughput

and gets the output in parallel across a physical page. Note that the data stored in 3D NAND may have a reliability issue, i.e., bit errors, and thus the error correction code (ECC) is widely adopted in 3D NAND [19], [39], [46], [74], [75], [76], [86]. The NAND controller takes time to process ECC right after data is read out from NAND chips to acquire the exact data. Generally, ECC is critical to the nvIMC design for VSS applications since it guarantees data reliability; nevertheless, ECC would also lead to high t_{SR} and E_{SR} due to the encoding and decoding procedures.

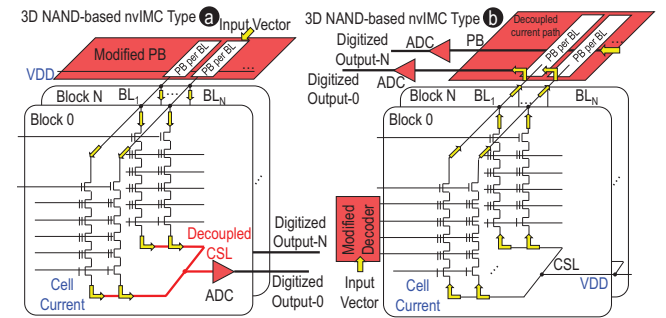


Figure 4: The concept of prior-art 3D NAND-based nvIMC works (with major modifications in red).

2) *Previous 3D NAND-based nvIMC Designs:* To meet the high memory density requirement and enhance the computing performance for VVM operations, researchers have proposed numerous 3D NAND-based nvIMC designs in recent years [17], [59], [63], [65], [92]. Fig. 4 illustrates two primary types of 3D NAND-based nvIMC designs for VVM operations.

Type a 3D NAND: It needs multiple decoupled common source lines (CSLs). Each one couples to one ADC. Once the VVM operation is triggered, each bit of the input vector controls the BL to generate current or not. All the unselected WLs in the other layers are biased at V_{PASS} , which makes the unselected transistors act as pass transistors. Meanwhile, the selected WL is forced to V_{READ} . The current may flow from the PB via decoupled CSL to ADCs for quantization. Hence, this solution needs to modify PB, break one CSL into pieces and add multiple ADCs to support VVM operations. For example, the prior work [65] deploys four ADCs to quantize the summation current separately and adds the four results together as a one-dimensional result. [17] also needs multiple

ADCs/DACs to enable VVM operations.

Type b 3D NAND: Once the VVM operation is triggered, the CSL is forced to VDD and the BLs are shorted to lower voltage. The unselected WLs are forced to V_{PASS} . Different from Type a, the V_{READ} or ground is applied to the WLs in the selected layer according to the input vector pattern. The current then flows from CSL via PB to ADCs for quantization. However, to independently quantize the summation current, this type of 3D NAND [59], [92] needs to break the PB's power line into pieces and add numerous ADCs at the BL-wise terminal. It would also increase the design overhead in WL-wise and BL-wise directions simultaneously.

To further measure and verify the overhead of these complicated circuit requirements, we conducted the experiments for evaluating the representative circuit requirement (i.e., ADC and DAC) by practically implementing the design of existing works [17], [65] on 96-layer 3D NAND with the circuit under array technology. The experiment results show that one 8-bit ADC (resp. DAC) contributes up to 2% (resp. 0.8%) power overhead and 0.04% (resp. 0.01%) area overhead over the 96-layer 3D NAND products. To realize INT8 VVM operations over 3D NAND, the number of ADC can be up to 2,048 [65], which would dramatically increase current and area overheads. As a result, adopting the 3D NAND-based mixed-signal nvIMC designs of existing works for VVM operations might be an inappropriate solution.

C. Motivation

This work is motivated by three challenges: (1) the long t_{SR} and high E_{SR} over edge devices for large feature vector datasets of VSS applications, (2) the tradeoff between VSS performance and adopting ECC or not, and (3) the significant power consumption and chip size overhead of existing 3D NAND-based nvIMC works for VVM operations. Four major technical concerns must be resolved before ICE becomes a feasible solution to empower VSS acceleration by exploiting 3D NAND-based nvIMC techniques. (1) How to realize digital nvIMC (i.e., ADC/DAC-free approach) in commercial 3D NAND without significant power, area overheads and complicated circuit designs in previous mixed-signal nvIMC designs, and also consider the cost and design minimization. (2) How to achieve high computing performance when processing a large amount of VVM computations over resource-constrained 3D NAND-based edge devices without the help of high DRAM density. (3) How to tolerate error bits in high-density 3D NAND (e.g., TLC) even without the support of ECC, which was missing considering the error-bit issue in previous works. (4) How to accomplish VVM operations over 3D NAND while maintaining decent accuracy, such as supporting signed (+/-) computations that were missing in the previous works. In the following sections, we take both software and hardware into design consideration. In particular, ICE with cognitive digital-based 3D NAND design for achieving high density, bandwidth, and parallelism will be presented.

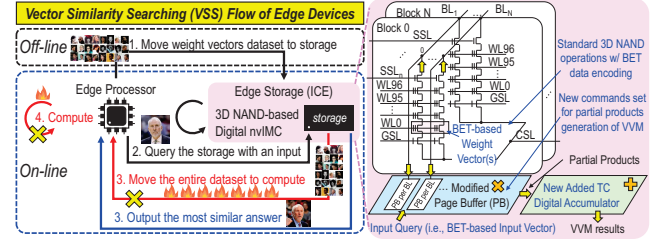


Figure 5: The concept of the edge device and ICE cooperation for VSS acceleration. The edge processor can be a feature extractor for input query and weight vectors generation. ICE can contribute to the offline phase on storing different datasets and the online phase for searching the most similar answer to suppress return data. ICE is with 3D NAND-based digital nvIMC techniques.

III. ICE WITH COGNITIVE 3D NAND

In this section, we will present our proposed ICE with cognitive 3D NAND. The goal of ICE with cognitive 3D NAND design is to enable high-performance VVM operations for VSS applications over edge devices by addressing the technical challenges and concerns mentioned in Sec. II-C. Specifically, our design aims to optimize the computing throughput, energy efficiency, reliability and accuracy over resource-constrained edge devices, by providing highly parallel VVM computation in cognitive 3D NAND with signed and low data precision (e.g., INT8 and INT4). First, We will show the architecture and workflow overview of ICE (see Sec. III-A). The proposed BET data encoding and circuit implementation for parallel bit multiplication and accumulation will be described (see Sec. III-B). After that, we will present the BET-based vector data management in ICE (see Sec. III-C). Finally, we will provide more example extensions for in-place logic computation supported by ICE (see Sec. III-D).

A. Architecture and Workflow Overview

As shown in Fig. 5, we propose the concept of system cooperation between the edge device and ICE for VSS acceleration. The edge processor (or edge GPU) can perform the feature extraction to extract feature vectors. In the offline phase, the processor extracts the feature vectors from the entire weight vector dataset and further moves all the extracted vectors to the 3D NAND chips in ICE. In the online phase, the processor first extracts the feature vector (i.e., the input vector) as an input query once it receives a raw input vector. ICE then utilizes the cognitive 3D NAND to score the similarity between the input query and all the weight vectors online. Finally, different from returning all the weight vectors to compute similarity scores in conventional processors, our proposed ICE only needs to return the most similar answer to the processor. Overall, ICE can support the storage of big high-dimensional feature vector datasets and

needs to broadcast each input once at the beginning of the entire search procedure.

B. BET-based VVM

This section presents the BET data encoding for tolerating error bits without the support of ECC during the VVM computation. Note that, in order to enable high-performance computation, ICE skips the conventional ECC only in the VVM computation procedure. However, ICE is still equipped with ECC for data reprogramming/refreshing to retain decent accuracy over the entire lifespan of 3D NAND (see Sec. III-C). After that, we will elaborate on how to realize digital BET-based¹ VVM using the modified PB and TC digital accumulator, as shown in Fig. 6. It aims at maintaining the high-density and low-bit-cost features of 3D NAND without sacrificing reliability.

To conduct the similarity search in ICE, we first apply the normalization procedure to all the input and weight vectors during the extraction stage. In this way, we could directly ignore the first two terms (i.e., $\|x\|^2$ and $\|w_i\|^2$) in Equation 2 since the length of all the normalized vectors are identical. That is, VVM becomes the major computing bottleneck for VSS as mentioned in Sec. II-A1. In the following sections, we assume that each value in the input and weight vectors are 8b integers. For instance, x_k can be represented as: $x_7, \dots, x_1, x_0$. Hence, $x_{k(i)}$ is the k^{th} bit of the i^{th} dimension of the input vector. $w_{k(i)}$ is the k^{th} bit of the i^{th} dimension of the weight vector.

1) *BET Data Encoding for Data Reorganization:* Since ICE has no shift-and-add units, it cannot conduct VVM computation by adopting the general binary data format. To tackle this issue, we propose the BET data encoding to encode data into the 2's complement-based unary format. The advantages of BET data encoding are threefold: (1) enabling VVM computation in ICE, (2) achieving bit-level parallelism by combining the modified PBs with the TC digital accumulator, and (3) enhancing data reliability even without the support of ECC. The rationale behind the reliability enhancement is that the number of each data bit is identical in the binary data format, and thus any error occurring in the most significant bits (MSBs) could cause severe accuracy degradation. On the contrary, since the unary data format duplicates each data bit based on its significance, we can still maintain decent accuracy even with a certain number of unary MSB errors. As shown in Fig. 6, after receiving feature vectors (i.e., input queries and weight vectors) in both offline and online phases, the ICE controller converts them into the BET-based vectors to suppress the bit-error influence. Even though the unary data format provides better data reliability, it trades data reliability with space utilization. In other words, we cannot arbitrarily leverage the unary format to store 8b binary-formatted integers (INT8).

¹Our proposed design can also conduct VVM computations in other integer-based data formats with minor design modifications.

For example, the unary format would require 255 NAND cells to store only an 8b integer, and it takes nearly $32\times$ space that of the binary format. To balance between data reliability and space utilization, the BET data encoding splits each binary-formatted integer into two unary-formatted integers. Fig. 7 illustrates the workflow of BET data encoding that uses 4b binary vectors. As shown in Fig. 7(a), for each 4b binary-formatted vector, the BET data encoding splits it into two 4b unary-formatted vectors, the even vector and the odd vector (a1). Note that even/odd vectors consist of half MSBs and half LSBs of the initial binary vectors, and each vector represents its 4b unary-formatted integers with 2/1 bit(s) from MSB to LSB, respectively. Meanwhile, for design uniformity, we add a spare bit storing zero to make the unary bit width being as the multiple of two. The even vector then merges the odd vector as the BET-based weight vector (a2). Compared to the conventional unary-formatted data encoding, our proposed BET data encoding can enhance space utilization while still protecting more significant bits with more unary bits.

To achieve INT4 multiplications, similar to existing works [44], our proposed BET also exploits digital circuits to produce partial products and processes shift-and-add functions sequentially. However, existing works take several cycles to finish the multiplications, and only few bits access requirements in every iteration, which would lead to low utilization against a large page size of 3D NAND and a large need to buffer all the partial products. Specifically, as shown in Fig. 7(b), BET duplicates each input bit 8 times to generate partial products by multiplying the repeated input bits by the 8b BET-based weight (b1). Besides, the multiplication results may need to be subtracted to keep the final results in the correct 2's complement format, as shown in Fig. 7 (b2) (the items marked in red). Nevertheless, implementing the subtraction function in the page buffer would lead to high design complexity at tight-pitch BL. Instead, BET aims at generating all the partial products of multiplications in merely a single access iteration at the page buffer by unfolding to enable high bit-level parallelism (b3). After unfolding, the partial multiplications then become multiple single-bit multiplications. To support the subtraction function, we then propose TC digital accumulator. TC features inverting the partial products and adding 1 with the relative weighting in the final results, which is equivalent to the conventional subtraction function. Moreover, to suppress the control complexity, BET puts the identical weighted partial products together for accumulating simultaneously, as shown in Fig. 7(c).

Once ICE is requested for BET-based VVM operations, ICE triggers page buffer to perform $64\times$ single-bit multiplication function in parallel to generate partial products. It is referred to as the bit-multiplication function, as shown in Fig 7(d). Finally, the TC digital accumulator performs the bit-accumulation function, as shown in Fig 7 (e). The

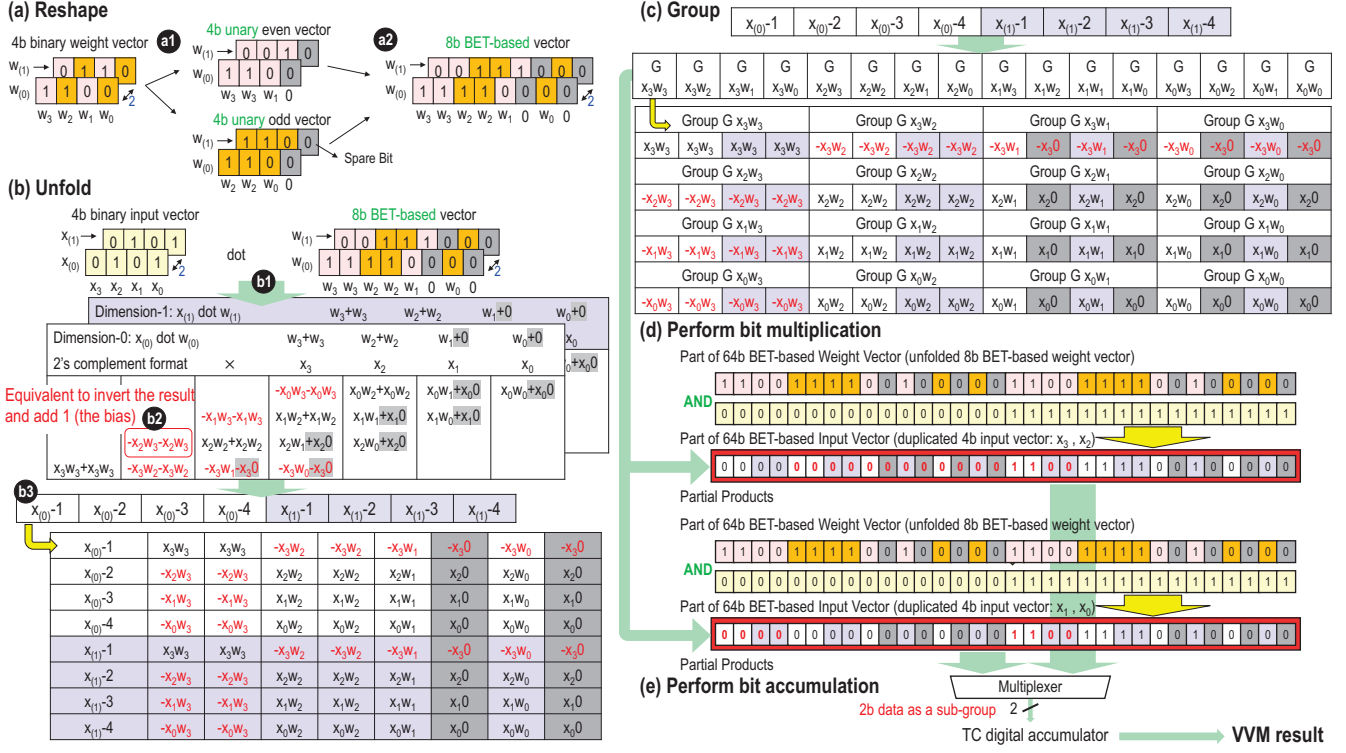


Figure 7: Bit-error-tolerance (BET)-based VVM flow. To achieve BET-based VVM, it needs five major steps. (a) Reshape the weight vector. (b) Unfold the multiplications. (c) Group the same weighted value. Then, perform (d) bit multiplication, and (e) bit accumulation to get the VVM result. Note that this is a short example for illustration, and ICE could support higher dimension and data precision vectors (e.g., 512-dimensional 8b vectors).

detailed concept and workflow of the digital accumulator will be demonstrated in Sec. III-B3. Note that the BET-based VVM can also be utilized on other signed and unsigned data formats with different data precision, e.g., INT8, UINT8, UINT4, etc.

2) Circuit Implementation for Bit Multiplication: We propose a combo sensing procedure that uses standard read and logic-AND functions, which are originally embedded in PBs of commercial 3D NAND for verifying procedures, so as to realize bit-level parallelism by adopting the BET data encoding. Therefore, our proposed design does not increase any area overhead of PBs but only has additional logic-AND latency after sensing. Fig. 6 (b) depicts the concept of the circuit. Specifically, the input data will be first sent into L2 through CDL before executing bit multiplication. At the beginning of bit multiplication, cognitive 3D NAND will access the selected page where the weight data are stored. Three read iterations using three V_{READ} levels for discriminating the middle page of the TLC (as mentioned in Sec. II-B1) will decode these partial read-out data into the accurate middle page data stored in DL (b1). Afterward, we will perform the logic-AND operation (b2) to “AND” the read-out data stored in DL via standard read operation and the input data and store the AND result in CDL.

3) Circuit Implementation for Bit Accumulation: Generally, a built-in counter in standard NAND products performs counting during program-verify procedures to check the program validation of each 4KB data. We extend the computing ability of this function, referred to as the fail-bit-count (FBC) function, to support unweighted accumulation directly using a 16KB-page size. Moreover, to achieve the BET-based VVM, we re-design the built-in counter as a TC digital accumulator with shift-and-add and inversion functions to accumulate the partial products. Fig. 6 (c) depicts the data flow and design concept. The TC accumulator consists of four steps shown in Fig. 8: (a) accumulate the same weighted partial products, (b) invert partial products and accumulate multiple partial products, (c) add bias to get the VVM result, and (d) record and output the result. The accumulator consists of an internal weighted-ACC control circuit, multiple logic function blocks, a counter, three accumulators, and multiple registers. In the first step, the accumulator is responsible for counting the value of 2b input of 16KB BLs (a1). TC then accumulates the partial sums, which are the partial products of the identical weighted value (a2). After that, TC extends the data width of the partial sums of the identical input weighted value and weights the values (b1). It then inverts the sums of the specific-needed partial products according to

(e) Perform bit accumulation

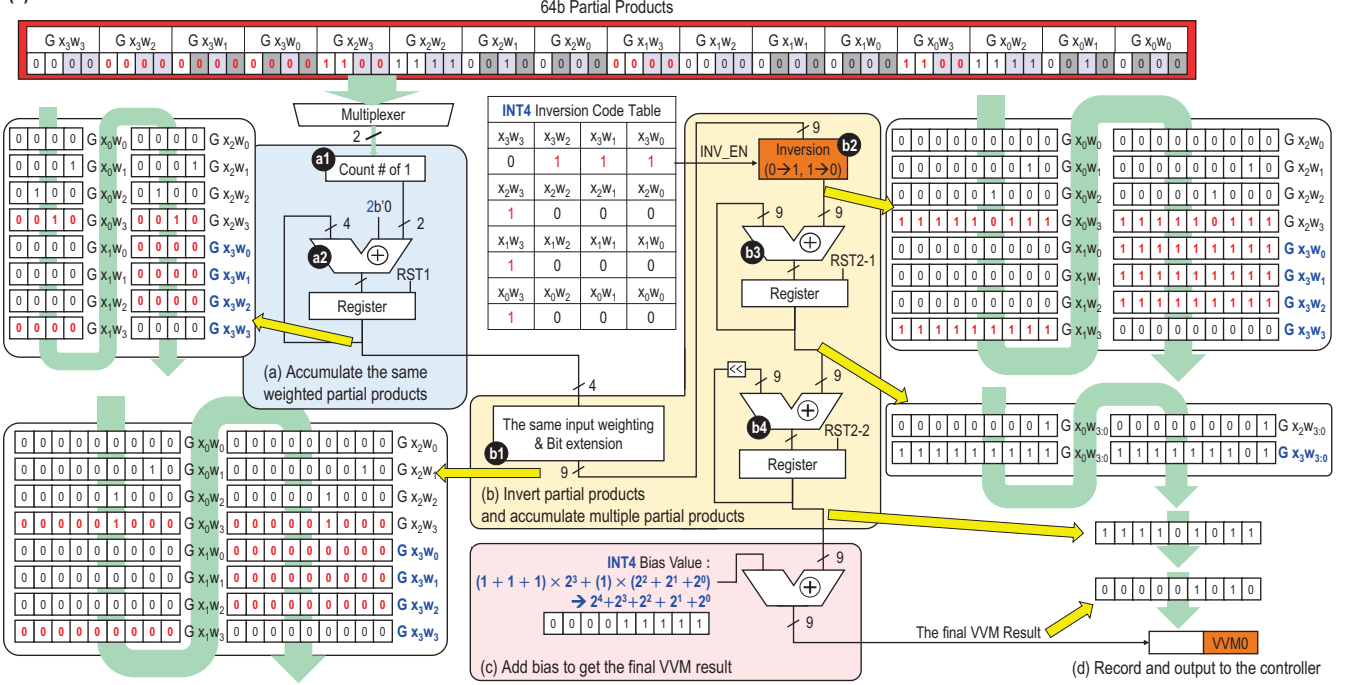


Figure 8: The concept and data flow of TC digital accumulator. Note that this is a short example for illustration. The proposed TC can support higher dimension and data precision vectors with higher input bandwidth and internal data width (e.g., 512-dimensional BET-based 8b VVM with 128b input bandwidth and 32b internal data width.)

the INT4 inversion code table (b2). TC then accumulates the sums, which are the identical input weighted values (b3), and performs shift-and-add to accumulate those different input weighted sums (b4). Afterward, TC adds the bias value (e.g., six different weighted “1” in the short example) according to the precision, aiming to keep the correct value in the 2’s complement format. Finally, ICE can access the result after storing it in the registers.

In practice, there is no additional area overhead of implementing bit multiplication but only the bit accumulation implementation. The small additional area overhead comes from a central control circuit, inversion functions, and registers for weighted accumulation. Therefore, the proposed digital nvIMC techniques will not violate the tight-pitch constraints of BL-wise and WL-wise directions. With such minor hardware modification, ICE can support the BET-based VVM for various data formats and precision without not only ECC overhead but also ADCs/DACs (see Sec. VI-A for a more detailed area breakdown). In addition, since ICE could reuse most of the hardware function blocks in standard 3D NAND, the cost of developing/fabricating ICE shall be limited.

C. Data Management in ICE

1) *Error Correction Code:* Previous works [65] only proposed their techniques on 3D single-level-cell (SLC)

NAND for VVM operations due to their operations of analog current summation. Our proposed digital nvIMC technique is allowed to store the weight vectors on 3D TLC NAND cells and achieves three times more data capacity than 3D SLC cells. Because the eight voltage states of TLC can represent three data bits, three logical pages are derived from a physical page. All three logical pages (i.e., low page, middle page, and high page) can store weight vectors. Although adopting the BET data encoding could remarkably mitigate the accuracy degradation due to significant bit errors, as mentioned in Sec. III-B1, it might still be insufficient to retain decent accuracy over the entire lifespan of NAND flash devices. It is necessary to reprogram the weight vectors periodically or occasionally to eliminate the severe accuracy degradation caused by NAND flash data retention. However, the reprogramming procedure has to get error-free weight vectors in advance. Therefore, executing ECC to protect weight vectors before reprogramming is essential for ICE. In this work, we adopt the quasi-cyclic low-density parity-check (QC-LDPC) code [55] with 4KB data and 512B parity as our ECC. As shown in Fig. 9, to avoid imprecise VVM operations, the TC digital accumulator will ignore the BLs that are parity bits during bit accumulation.

2) *Data Precision:* There is a tradeoff among similarity search speed, accuracy, and data precision in ICE. By adopting the BET data encoding and the data layout shown

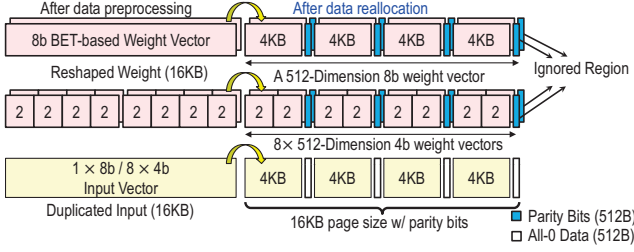


Figure 9: Data mapping of the physical page for parity bits and all-0 data. The partial products of these regions would be zero, maintaining the VVM results without influence.

in Fig. 9, a BET-based vector with 8b precision occupies 131,072 ($2^4 \times 2 \times 512 \times 8$) bits for 8b weight value, 512-dimension feature vectors, and 8b input value. If we reduce the precision to 4b, the size of each vector reduces to 16,384 ($2^2 \times 2 \times 512 \times 4$) bits, allowing us to support $8\times$ the number of vectors in a single page. That is, the performance of ICE can be further enhanced by leveraging low-precision inputs/weights according to the accuracy need for different VSS applications.

3) *Support of Storage and VSS:* Our proposed design exploits NAND chips for storage and VSS simultaneously. ICE can be realized on edge storage products, e.g., embedded Multi-Media Card (eMMC). Due to the growing capacity of NAND chips, it shall have sufficient space to keep user data and feature vectors together in cognitive 3D NAND. Besides, we assume a software management layer, i.e., flash translation layer (FTL), in the ICE controller. The primary goal of FTL is to address translation which maps logical block addresses (LBAs) of every access request (e.g., read and write) into physical block addresses (PBAs) in 3D NAND flash memory space. ICE is proposed to separate the memory space into two different addressing types, one for VSS applications addressed by 16KB-based LBA and the other for standard data storage addressed by 4KB-based LBA [2], [6]. We also assume that the FTL adopts multiple essential components, including the page-level FTL mapping scheme [103], greedy garbage collection scheme [13], and static wear-leveling scheme for lousy block management [24]. As for the problem of optimizing these software/firmware schemes, we leave it as future work.

D. Extension for In-Place Logic Computation

Cognitive 3D NAND also provides different in-place logic operations in PBs, e.g., NOT, XOR, OR. Leveraging these operations allows ICE to support not only the similarity search but also other computations, e.g., Hamming distance [68], [101], fully connected neural networks (FCNN), and ternary content addressable memory (TCAM) in memory-augmented neural networks (MANN) [49], [61]. For instance, ICE can directly store the weight patterns in cognitive 3D NAND to support Hamming distance computations. It does not

need to adopt the BET data encoding to protect data since each bit is a stand-alone item in Hamming distance. Once receiving the input query, ICE would broadcast it into L2 of each PB of cognitive 3D NAND chips. ICE then triggers 3D NAND to perform the standard sensing operation and stores weight data in DL, as shown in Fig. 6 b1). After that, cognitive 3D NAND performs the logic-XOR function of the weight data and the input data and stores the result in CDL, as shown in Fig. 6 b3). Finally, cognitive 3D NAND processes unweighted bit-accumulation to generate the Hamming distance result. Cognitive 3D NAND can also directly perform our proposed VVM to support FCNN, which needs a high-dimensional inner product. By combining two BLs as a pair, performing standard sensing, logic-AND operations, and unweighted bit accumulation, cognitive 3D NAND can also support TCAM in MANN.

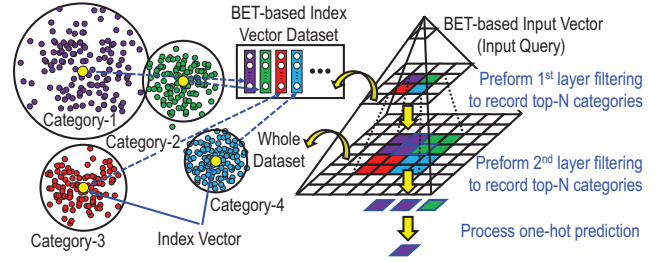


Figure 10: The concept of hierarchical top-N search scheme.

IV. ICE RUNTIME SYSTEM

In this section, we will focus on the runtime system and software-hardware interface framework to support ICE. First, we will present the high-level overview of our system and its critical component (i.e., the hierarchical top-N search scheme) to demonstrate how to optimize the system-level design (see Sec. IV-A). Then, the offline software-hardware interface (see Sec. IV-B) and online software-hardware interface (see Sec. IV-C) will be presented.

A. Hierarchical Top-N Search Scheme

As mentioned in Sec. III-A, the workflow of ICE for VSS applications primarily comprises two phases, the offline phase, and the online phase. (1) In the offline phase, ICE transforms the raw weight vectors into the BET-encoded format and stores all the BET-based weight vectors in cognitive 3D NAND. (2) In the online phase, once receiving a raw input query, ICE executes VSS, scoring the similarities between the input query and each weight vector and providing the best match. However, using brute force to calculate the similarity of the entire feature vector dataset results in long t_{SR} and high E_{SR} . Hence, we propose the hierarchical top-N search (HTNS) scheme to accelerate the search procedures without sacrificing accuracy in the ICE runtime system. As shown in Fig. 10, HTNS is composed of three layers to achieve a layerwise similarity search. In the first layer, HTNS conducts

Table I: ICE API Table.

API	Description
<i>Converge_V</i> (V_{RAW})	Converge V_{RAW} dataset to IV_{RAW} dataset.
<i>Quantize</i> (V_{RAW})	Quantize V_{RAW} to integer-based V_Q .
<i>Convert_BET_Weight</i> (V_Q)	Split and flatten V_Q to BET-based V_{BET} .
<i>Program</i> ($V_{BET}, addr$)	Program V_{BET} into 3D NAND with <i>addr</i> .
<i>Build_Table</i> (IV_{BET}, V_{BET}, ID)	Build a link table mapping the relationship between IV_{BET} , V_{BET} and ID .
<i>Build_Dictionary</i> ($ID, Info$)	Build a dictionary mapping the relationship between ID and $Info$.
<i>Convert_BET_Input</i> (IQ_Q)	Duplicate each bit of IQ_Q to BET-based IQ_{BET} .
<i>Set_HTNS</i> (N_1, N_2)	Setup N_1 and N_2 of HTNS for sorting top- N_1 categories in the first layer and top- N_2 categories in the second layer.
<i>Sort_TopN_ID</i> (<i>score</i> , N_1, T)	Sort similarity <i>score</i> results and return Top- N_1 categories by referring to T .
<i>Partition</i> (IV_{BET}, M)	Partition IV_{BET} into <i>count</i> groups, where each group consists of M vectors.
<i>Execute_VVM</i> ($IQ_{BET}, IV_{BET}, M, index$)	Execute BET-based VVM with IQ_{BET} and M vectors in IV_{BET} .
<i>Retrieve_V</i> (V_{BET}, Top_ID_{L1}, T)	Retrieve a set of vectors in V_{BET} by referring to T and Top_ID_{L1} .
<i>One_Hot_Prediction</i> (Top_ID_{L2}, T)	Conduct one-hot prediction to find the most similar category in Top_ID_{L2} by referring to T .
<i>Retrieve_Info</i> (<i>Dic</i> , <i>id</i>)	Retrieve the detailed information by referring to <i>Dic</i> and <i>id</i> .
<i>Suspend_VSS</i> ()	Suspend the VSS operation, and record current search status and Q_{BET} into global data buffer.
<i>Resume_VSS</i> ()	Read out stored search status and Q_{BET} from global data buffer, and resume the VSS operation.
<i>Output_TopN</i> (N_1)	Output the indices of top- N_1 categories and their detailed information.
<i>Execute_Single_Plane_VVM</i> (IQ_{BET}, IV_{BET})	Execute the single-plane BET-based VVM operation with IQ_{BET} and IV_{BET} .
<i>Execute_Four_Plane_VVM</i> (IQ_{BET}, IV_{BET})	Execute the four-plane BET-based VVM operation with IQ_{BET} and IV_{BET} .

the coarse-grained search to obtain the top-N categories that are most similar to the input query. Instead of comparing the similarity of the input query with the entire weight vector dataset, here we only compare the input query with a single “index vector” for each category. Furthermore, in the offline phase, the processor in ICE averages every dimension of all the weight vectors in each category as an index vector, as shown in Fig. 10. By this means, we can significantly reduce t_{SR} and E_{SR} in the online phase. In the second layer, HTNS processes the fine-grained search through all the weight vectors of the top-N categories which were acquired in the first stage. Finally, HTNS picks the most similar answer by applying the one-hot prediction to the results obtained in the second layer.

B. Offline Software-Hardware Interface

Fig. 11 shows the stack of ICE for supporting VSS programming, which allows developers to deploy ICE in VSS applications. A detailed explanation of each application programming interface (API) can be found in Table I. In the programming stage of the offline phase, ICE provides APIs so that it allows developers to (1) generate the BET-based index and weight vectors, *Convert_BET_Weight*() (①), (2) generate the link table and information dictionary, *Build_Table*() (①) and *Build_Dictionary*() (②), and (3) program the BET-based index and weight vectors into cognitive 3D NAND, *Program*() (③). When updating the feature vectors which were already stored in cognitive 3D NAND, ICE reuses the identical APIs to allow developers to (1) use *Convert_BET_Weight*() (①) to generate the new BET-based index and weight vector and (2) update the weight vectors and index vectors stored in cognitive 3D NAND,

Algorithm 1: OFFLINE: BUILD AND STORE DATASET

Input: V_{RAW} : a dataset of raw weight vectors. ID : a set of integers indicating the index of each category. $Info$: a set of strings indicating the detailed information of each category.

Output: IV_{BET} : the BET-based index vector dataset. V_{BET} : the BET-based weight vector dataset. T : a link table mapping the relationship between IV_{BET} , V_{BET} , and ID . *Dic*: an information dictionary mapping ID and $Info$.

```

1 begin
  // Converge and quantize dataset
2    $IV_{RAW} \leftarrow Converge\_V(V_{RAW})$ ;
3    $IV_Q \leftarrow Quantize(IV_{RAW})$ 
4    $V_Q \leftarrow Quantize(V_{RAW})$ 
  // Store dataset in 3D NAND
5    $IV_{BET} \leftarrow Convert\_BET\_Weight(IV_Q)$ 
6   Program( $IV_{BET}, addr$ )
7    $V_{BET} \leftarrow Convert\_BET\_Weight(V_Q)$ 
8   Program( $V_{BET}, addr$ )
  // Prepare metadata
9    $T \leftarrow Build\_Table(IV_{BET}, V_{BET}, ID)$ 
10  Dic  $\leftarrow Build\_Dictionary(ID, Info)$ 
11 end

```

Program() (③).

The concept of the offline APIs in ICE can be better explained by Algorithm 1. The goal of Algorithm 1 is to build and store the index and weight vector dataset into cognitive 3D NAND and prepare additional metadata (i.e., link table and information dictionary) that will be exploited in the online phase. We assume there is a dataset of raw weight vectors V_{RAW} . ID is a set of integers indicating the index of each category in V_{RAW} , and $Info$ is a set of strings indicating the detailed information of each category.

Converge and quantize the dataset: In Line 2, the processor converges the raw weight vector dataset (V_{RAW}) into a relatively small dataset of index vector (IV_{RAW}) by averaging every dimension of all the raw weight vectors in each category. In Lines 3–4, the processor quantizes the raw vectors (V_{RAW} and IV_{RAW}) into integer-based vectors (V_Q and IV_Q).

Store the dataset: In Lines 5–8, ICE converts the quantized index and weight vectors into the BET-based encoding format and further stores/programs them in cognitive 3D NAND chips with specific addresses (*addr*).

Prepare metadata: In Lines 9–10, ICE builds a link table (T) and an information dictionary (*Dic*). T is used to map the relationship between IV_{BET} , V_{BET} and ID , and *Dic* is used to map the relationship between ID and $Info$. T and *Dic* will be used in the online phase.

C. Online Software-Hardware Interface

In the programming stage of the online phase, ICE provides APIs for developers to (1) generate BET-based input query for executing VSS applications, *Convert_BET_Input*() (④), (2) setup parameters of HTNS, e.g., top-N of each level,

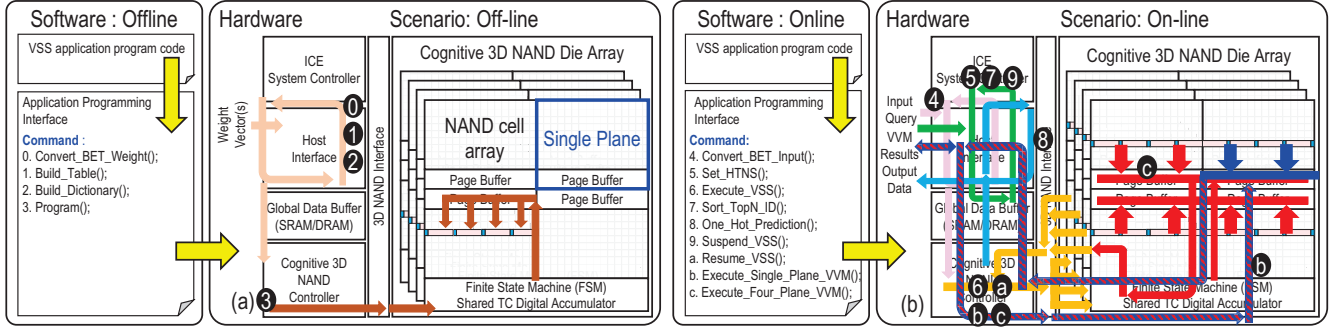


Figure 11: ICE software-hardware interface framework. (a) In the offline phase, ICE processes APIs to update a BET-based weight vector dataset and a BET-based index vector dataset. (b) In the online phase, the system processes APIs to convert a raw input vector into a BET-based input vector and performs hierarchical top-N search scheme to search for the most similar answer.

Algorithm 2: ONLINE: EXECUTE HTNS-BASED VSS

Input: IQ_{RAW} : a raw input query. T : a link table mapping the relationship between IV_{BET} , V_{BET} , and ID . Dic : an information dictionary mapping ID and $Info$. IV_{BET} : the BET-based index vector dataset. V_{BET} : the BET-based weight vector dataset. M : the number of VVM results per iteration. N_i : the number of top categories in i^{th} layer of HTNS.

Output: id : the index of the most similar category. $info$: the detailed information of the most similar category.

```

1 begin
2    $IQ_Q \leftarrow Quantize(IQ_{RAW})$ 
3    $IQ_{BET} \leftarrow Convert\_BET\_Input(IQ_Q)$ 
4    $Set\_HTNS(N_1, N_2)$ 
5    $count \leftarrow Partition(IV_{BET}, M)$ 
   /* First layer of HTNS: coarse-grained search */
6   for  $i$  in  $range(0, count - 1)$  do
7      $index \leftarrow i * M$ 
8      $score \leftarrow Execute\_VVM(IQ_{BET}, IV_{BET}, M, index)$ 
9   end
10   $Top\_ID_{L1} \leftarrow Sort\_TopN\_ID(score, N_1, T)$ 
11   $V'_{BET} \leftarrow Retrieve\_V(V_{BET}, Top\_ID_{L1}, T)$ 
12   $count \leftarrow Partition(V'_{BET}, M)$ 
   /* Second layer of HTNS: fine-grained search */
13  for  $i$  in  $range(0, count - 1)$  do
14     $index \leftarrow i * M$ 
15     $score \leftarrow Execute\_VVM(IQ_{BET}, V'_{BET}, M, index)$ 
16  end
17   $Top\_ID_{L2} \leftarrow Sort\_TopN\_ID(score, N_2, T)$ 
   /* Third layer of HTNS: one-hot prediction */
18   $id \leftarrow One\_Hot\_Prediction(Top\_ID_{L2}, T)$ 
19   $info \leftarrow Retrieve\_Info(Dic, id)$ 
20  return  $id, info$ 
21 end

```

$Set_HTNS()$ (5), (3) start to execute VSS operation, $Execute_VSS()$ (6), (4) sort the top-N categories, $Sort_TopN_ID()$

(7), (5) process one-hot prediction to find the most similar answer within the top-N categories, $One_Hot_Prediction()$ (8), (6) record the current status information into the global data buffer to suspend the VSS operation after receiving the requests from the host, $Suspend_VSS()$ (9), (7) resend BET-based input query to cognitive 3D NAND and re-execute the VSS operation, $Resume_VSS()$ (a). Note that the data duplication approach, $Convert_BET_Input()$ (4), has an almost negligible negative impact since we only have to conduct the duplication process once during the entire online phase.

Our proposed design does not need to write back any data to NAND chips during search procedures. The data buffer can store the intermediate data, e.g., the top-N data during an online search or the address of current status after receiving suspend command, which is relatively small and can be directly stored in the data buffer. Moreover, considering the general purpose of neural network computations, users can have the flexibility to command cognitive 3D NANDs with single or multiple planes directly, i.e., the single-plane VVM operation $Execute_Single_Plane_VVM()$ (b), and the four-plane VVM operation $Execute_Four_Plane_VVM()$ (c).

In Algorithm 2, we execute the HTNS for vector similarity search and eventually output the most similar answer. We assume the BET-based index and weight vector dataset (i.e., IV_{BET} and V_{BET}) are already stored in cognitive 3D NAND chips. Besides, there is a link table T mapping the relationship between IV_{BET} , V_{BET} , and ID . The information dictionary Dic maps the relationship between ID and $Info$.

Preprocessing: In Lines 2-3, ICE quantizes the raw input vector/query and further converts it into the BET-based input vector. In Line 4, ICE sets up the HTNS parameters, e.g., the number of top categories in each layer of HTNS (N_1 and N_2).

HTNS Execution: In Line 5, ICE partitions the index vector dataset (IV_{BET}) into $count$ groups. Each group consists of M vectors. Considering the die-level and plane-level

parallelisms, M can be different based on the hardware setting, such as the number of 3D NAND chips and the number of planes in each chip. In Lines 6-9, ICE executes the first-layer search in HTNS. Specifically, ICE triggers the BET-based VVM on cognitive 3D NAND in parallel and obtains the similarity scores. In Line 10, ICE acquires top- N_1 categories of the first-layer HTNS (Top_ID_{L1}) by sorting the similarity scores. In Lines 11-12, ICE picks the weight vectors of specific categories (V'_{BET}) based on Top_ID_{L1} , and partitions them into *count* groups. In Lines 13-16, ICE executes the second-layer search in HTNS and obtains the similarity scores. In Line 17, ICE acquires top- N_2 categories of the second-layer HTNS (Top_ID_{L2}) by sorting the similarity scores. In Lines 18-19, ICE conducts the one-hot prediction to get the most similar category (*id*), and retrieve the detailed information (*info*) in *Dic*. Finally, cognitive 3D NAND returns the most similar category and related information to the host controller in Line 20.

V. EXPERIMENT SETUP

We adopt the face search application as our experimental study for VSS applications. Face search is used to distinguish whether a pair of faces belong to the same person or not within a dataset with discriminating features. Our proposed design, vector similarity search, still remains general and can be applied to various implementations/applications, e.g., access control, image classification [60], [73], object recognition [36], [53], [69], speech recognition [14], [87], etc.

A. Workload

We implement face search application on the open-source CelebA dataset [64], VGGFace2 [21], and CASIA-WebFace [100]. We leverage a well-trained feature representation network model trained with triplet loss [26], [78] using the Pytorch [72] and Tensorflow framework [11] for feature vector extraction on various edge devices. Facial feature vectors can be projected in a 512-dimension space by adopting L2-normalization [26], [78] under a fixed vector length constraint. Then it will be quantized from FP32 to INT8 before VSS operations in ICE [16]. The original feature vector dataset in VGGFace2 (resp. CelebA, CASIA-WebFace) requires 340MB (resp. 40MB, 1GB) of memory capacity with 170K (resp. 20K, 500K) faces of 500 (resp. 1,000, 10,575) individuals. After adopting the BET data encoding, the 8b feature vector dataset in VGGFace2 (resp. CelebA, CASIA-WebFace) requires 680MB (resp. 80MB, 2GB) of memory capacity. The 8b index dataset of VGGFace2 (resp. CelebA, CASIA-WebFace) requires 0.4MB (resp. 1MB, 10.8MB) of memory capacity with 500 (resp. 1,000, 10,575) index vectors.

B. Systems for Benchmark

We implement the VSS applications on three edge systems for comparison, e.g., Raspberry Pi 3, Raspberry Pi 4, and NVIDIA Jetson Nano. The monitored power consumption

Table II: ICE and Cognitive 3D NAND Test Chip Spec.

3D NAND Technology Circuit-under-Array (CuA)		512Gb 96-Stacked-WL 3D NAND			
Main Array		512 Gb (TLC)			
Chip mode		VVM/Storage			
Plane		4			
Page Size		16 KB			
Block Size		18432 KB + 2304 B			
Operation Definition	Time (μ s)	VDD (V)	Current (A)	Energy (μ J)	
1-Plane SLC Read	50	3.3	0.045	7.43	
1-Plane TLC Read	70	3.3	0.046	10.63	
4-Plane SLC Read (Parallel)	50	3.3	0.062	10.23	
4-Plane TLC Read (Parallel)	70	3.3	0.067	15.48	
4-Plane Bit Multiplication (Parallel)	72	3.3	0.067	15.9	
4 $\times$ Bit Accumulation (Sequential)	140	3.3	0.0152	7.02	
Avg. per VVM w/ 4-Plane Parallelism	53	3.3	0.0328	5.74	
Component		Power		Unit	
2 $\times$ Controllers		158		mW	
128KB SRAM Buffer		105		mW	
3D NAND Interface		80		mW	
4 $\times$ Cognitive 3D NAND chips		884		mW	
ICE Total Power (Searching)		1227		mW	

of these systems is close to [1], [7]. For the device-level evaluation of VVM, we use HSPICE [31] for post-layout simulation to measure the energy consumption of modified circuits and performance. Moreover, we implement a cognitive 3D NAND test chip on 96-layer 3D NAND with the circuit under array (CuA) technology. A 3D NAND test chip measures the validations of the proposed functions. The silicon data of the test chip is shown in Table II.

VI. EXPERIMENT RESULTS

In this section, we will present a wide variety of experimental results to demonstrate both the feasibility and capability of our proposed design. First, we will show the details of our Cognitive 3D NAND test chip and further compare it with two prior 3D NAND-based nvIMC works in Sec. VI-A. In Sec. VI-B, we will demonstrate the accuracy results under various datasets and encoding scenarios, so as to further verify the feasibility and capability of ICE. Lastly, we will compare the total execution time and energy efficiency of three edge systems equipped with our proposed ICE and prior work in Sec. VI-C.

A. Implementation of Cognitive 3D NAND

We implement a cognitive 3D NAND test chip using the CuA technology for verifying our proposed VVM operation shown in Fig. 12(a). We also implement an 8b ADC on the test chip just for comparison, and its structure is referred to the work [82]. Fig. 12(b) shows the layout of the 8b ADC. Through measurement, we observe the ADC increases 0.04% the area overhead of the 3D NAND test chip. Work in [65] needs four planes with 2,048 ADCs to achieve a single UINT8 VVM operation in one read cycle, or one plane with 512 ADCs in four read cycles. Fig. 12(c) demonstrates the shared accumulator. The area overhead of the modified accumulator is 1.6% of the test chip. It proves that our proposed design is more feasible in commercial 3D NAND chips.

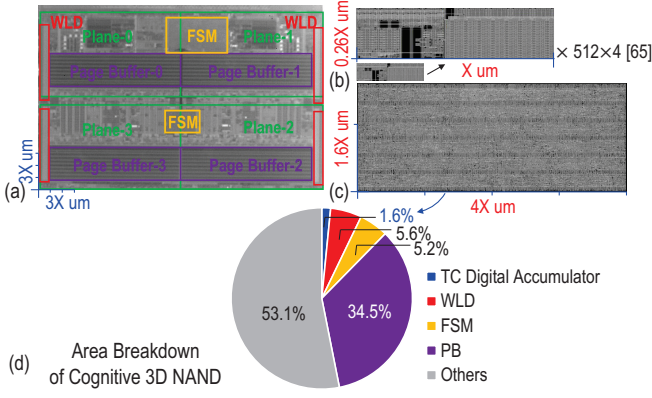


Figure 12: (a) Die photo (backside) of Cognitive 3D NAND test chip. (b) 8b ADC. (c) TC digital accumulator. (d) Area breakdown of wordline driver (WLD), finite state machine (FSM), page buffer (PB), and other peripheral circuits.

Table III: Comparison of Prior Arts and This Work.

Precision	This Work	IEDM 2019 [65]		IEDM 2018 [63]	
	INT8/INT4	UINT4	UINT8 (Re-de.)	1b	UINT8 (Re-de.)
Type	Digit	Current		Voltage	
Level of Cell	TLC	SLC		SLC	
Dimension #	512	512		512	
Total ADC #	0	N/A	512×4	N/A	512×16
Accumulator #	1	1		1	
I_{ANA} (uS)	N/A	1	50	2	50
I_{MUL} (uS)	72 (4×VVM)	N/A		N/A	
I_{ACC} (uS)	140 (4×VVM)	N/A	4	N/A	8
$I_{AVG-VVM}$ (uS)	53	N/A	54	N/A	58
Output Precision	Full Precision	N/A	14b	N/A	15b
Per ADC (mW)	N/A	2.57(4b)	3.3(8b)	2.57(4b)	
WLD (mW)	33 (4-plane)	33 (4-plane)		33 (4-plane)	
Page Buffer & Array (mW)	112 (4-plane)	0.0059	0.149	0.0026	0.541
FSM (mW)	23.6	23.6		23.6	
ACC (mW)	26.4	N/A	8	N/A	8
GCR (mW)	N/A	N/A		11	
Others (mW)	49.5	49.5		49.5	
Avg. P/VVM (mW)	107.5/26.9	N/A	6872.6	N/A	21,179
Avg. E/VVM (uJ)	5.7/1.4	N/A	371.1	N/A	1228.4
Additional Area Overhead (%)	1.6	137		546	

*Note: Analog Computing (ANA), Multiplication (MUL), Accumulation (ACC), Global Current Regulator(GCR), Average (Avg.), Power (P), Energy (E), Re-design (Rede.)

Table III compares the prior arts [63], [65] (analog solutions), and our work (digital solution). Note that, we did not really implement the works [63], [65], and their results in Table III are the simulation data based on HSPICE. Even though the previous works show the strength of analog computing at page buffer and cell array compared to our solution, they ignore the overhead of data initialization. Thus, these works need to be redesigned to generate VVM results of 512-dimension vectors. Generally, 3D NAND should perform a read-like operation to initialize the analog information before computing, e.g., stabilizing cell current at biased V_{BL} or stabilizing sensing current [65]. Hence, their proposed computing latency should be re-evaluated.

Moreover, the amount of ADCs they need to accomplish a single UINT8 VVM operation should also be re-calculated. After redesigning the work [65], its ADC number is 4 with a single UINT8 multiplication decomposition into four planes. Furthermore, each 8b ADC quantizes 15×15 cell current, two unsigned unary 4b integer, as an 8b result. After redesigning the work [63], it needs 8×2 with a single UINT8 multiplication decomposition into four planes. Besides, each 4b ADC quantizes the result of 1b input and 4b weight multiplication in one NAND string. Finally, they could trigger digital adders to accumulate these partial products and generate the VVM result of one 512-dimension vector. Note that we help them synthesize the digital adder with 128b input to evaluate the accumulation latency, taking roughly 4us [65] and 8us [63] under 30ns clock cycle time.

The simulation results show that our proposed solution can significantly enhance energy efficiency. Compared to the work [65] and work [63], the energy efficiency of UINT8 VVM computations of our work improves by $65 \times$ and $226 \times$, respectively. Moreover, ICE will not sacrifice the output precision (25b), where the work [65] is 14b, and the work [63] is 15b. Last but not least, our design is more like a commercial product for mass production since it only needs limited modification on peripheral circuits of standard 3D NAND products. It is worth noting that *this work is the first demonstration of digital in-NAND computation with silicon data.*

Table IV: Comparison of Simulated Accuracy.

Top-1 Acc.	CelebA(20K)				
	FP32	INT8	UINT8	INT4	INT2
Baseline	94.7%	94.7%	94.2%	94.7%	92.6%
w/ HTNS	95.1%	95.1%	94.4%	94.8%	93.6%
	VGGFace2(170K)				
	FP32	INT8	UINT8	INT4	INT2
Baseline	93.2%	93.2%	92.9%	93.2%	90.9%
w/ HTNS	93.7%	93.7%	93.1%	93.5%	91.9%
	CASIA-WebFace(500K)				
	FP32	INT8	UINT8	INT4	INT2
Baseline	82.6%	82.6%	82.6%	82.2%	77.4%
w/ HTNS	85.7%	85.7%	86.7%	85.5%	83.8%

B. Validation of Accuracy and Encoding

Table IV shows that the baseline accuracy (FP32) of CelebA, VGGFace2, and CASIA-WebFace are 94.73%, 93.23%, and 82.6%. After using the HTNS scheme, the accuracy (FP32) of CelebA, VGGFace2, and CASIA-WebFace can be slightly improved. We also simulate different data formats, i.e., INT8, UINT8, INT4, and INT2. Note that we only simulate the INT-based data formats which ICE supports. The results show that the UINT8, INT8, and INT4 data formats could maintain comparable accuracy results compared to FP32, while the accuracy of the INT4 data format slightly drops. The experiment results verify that our proposed design will not sacrifice accuracy.

In order to verify the effectiveness of our proposed BET data encoding, we then conduct an experiment to compare

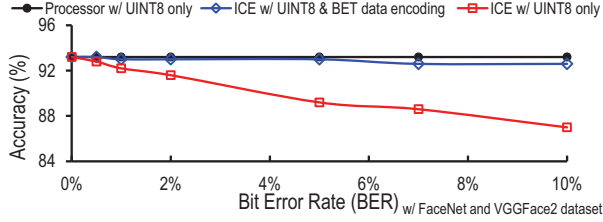


Figure 13: Simulated accuracy with different BER. The evaluated model is FaceNet, and the dataset is VGGFace2.

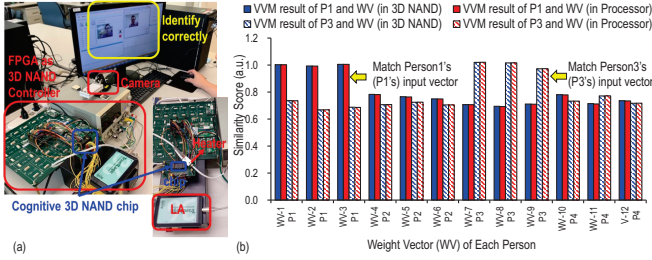


Figure 14: (a) The proof-of-concept VSS system. (b) Measured similarity scores. ICE adopts BET-based UINT8 data format, and the processor adopts the general UINT8 data format.

the accuracy under different BER with or without adopting BET data encoding. As shown in Fig. 13, the accuracy with adopting BET data encoding on ICE can be enhanced to be comparable with that on general processors. This is because BET data encoding encodes data in the unary format, duplicating data bits, especially on MSBs, so our proposed design can be significantly more error-tolerable.

We further built a proof-of-concept VSS system, shown in Fig. 14(a), measuring the validation of UINT8 VVM. Fig. 14(b) shows the measured similarity scores (i.e., VVM results) of using cognitive 3D NAND to compute BET-based UINT8 data and using a processor without the BET method. The x-axis is four persons' weight vectors, each with three weight vectors. The similarity scores using our solution are almost the same as the processor, and the most similar person has much higher scores than others. Using cognitive 3D NAND for scoring similarity could mitigate the ECC overhead even with a minor computing mismatch. Therefore, we have the strength to emphasize that using our solutions to implement INT8/INT4 VVM is effective.

C. System Comparison

Fig. 15 and Fig. 16 compare the total execution time and energy consumption using three basic edge systems; each edge system contains an ICE with four cognitive 3D NAND chips, or an ICE with four chips using the previous work [65]. To achieve a fair system and hardware comparison, we help [65] adopt our proposed HTNS in this experiment. With the configuration of the heavyweight AI model (FaceNet) and the small dataset (CelebA), these edge devices using ICE with

nvIMC techniques can enhance the execution time by $1.01\times$ - $1.4\times$ and the energy efficiency by $1.02\times$ - $3.3\times$. With the configuration of the lightweight AI model (MobileFaceNet) and the CelebA dataset, using ICE can enhance the execution time by $1.04\times$ - $1.4\times$ and the energy efficiency by $1.02\times$ - $6\times$. On the other hand, with the configuration of FaceNet and the VGGFace2 dataset, these edge devices using ICE can improve the execution time by $1.9\times$ - $7.6\times$ and the energy efficiency by $1.7\times$ - $5.1\times$. With the configuration of MobileFaceNet and the VGGFace2 dataset, using ICE can enhance the execution time by $1.7\times$ - $11.9\times$ and the energy efficiency by $1.4\times$ - $10.5\times$.

On the contrary, work in [65] needs many ADCs to accomplish VVM operations. It leads to significant energy consumption and a large area overhead compared to our proposed solution shown in Fig. 16. With the configuration of MobileFaceNet and the CASIA-WebFace dataset, using the previous work [65] improves the execution time by $16.9\times$ - $90\times$ and the energy efficiency by $10.11\times$ - $42\times$. With the same configuration, using ICE can improve the execution time by $17\times$ - $95\times$ and the energy efficiency by $11\times$ - $140\times$. Considering the overall system performance, the speed (resp. energy efficiency) of ICE is higher than that of [65] by $1.01\times$ - $1.05\times$ (resp. $1.03\times$ - $3.30\times$). Note that, without our proposed HTNS for [65], the speed (resp. energy efficiency) of ICE can significantly win [65] by $7\times$ - $148\times$ (resp. $9\times$ - $487\times$). Furthermore, our proposed ICE also outperforms [65] in other evaluation metrics such as area ($2.33\times$ smaller) since [65] has to deploy numerous ADCs. To sum up, using the lightweight MobileFaceNet can decrease t_{FE} , which may increase the percentage of t_{SR} with a large dataset. Hence, ICE decreases long t_{SR} and high E_{SR} on edge devices. Although we use the INT8 format here, it is reasonable to infer that ICE can still outperform other works in different INT-based formats, such as INT4.

For other applications with large datasets, the data movement and swap deteriorate the overall performance. At the same time, DRAM capacity is limited, as shown in Fig. 15 and Fig. 16. This phenomenon and characteristic are significant for our target environment, commonly resource-constrained edge devices. To address these issues, ICE could serve in a computational storage unit to compute data locally without triggering additional data movement and swap. The proposed ICE with cognitive 3D NAND can process the similarity search in parallel with computing units to improve overall performance, while the processors in edge systems can utilize DRAM completely for other complicated computations.

VII. RELATED WORK

Volatile Memory-based IMC: Works in [18], [23], [35], [38], [40], [54], [58], [82], [83], [85], [94], [99] aim to perform VVM in the memory for decreasing data movement. However, these volatile memory-based IMC solutions still need to move the data from storage after power-on. Works

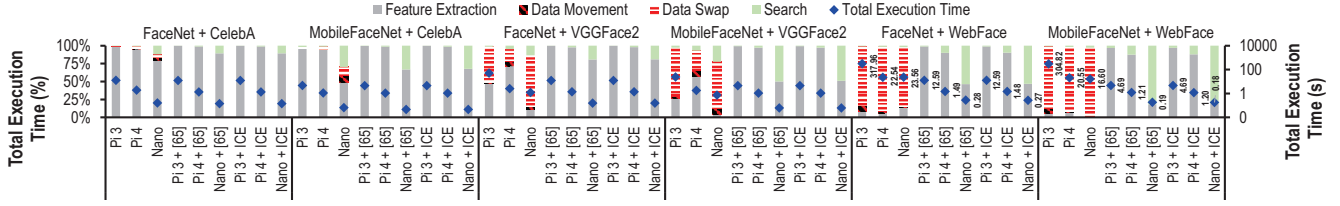


Figure 15: Comparison of the measured total execution time of edge systems and simulated total execution time of systems with [65] and ICE.

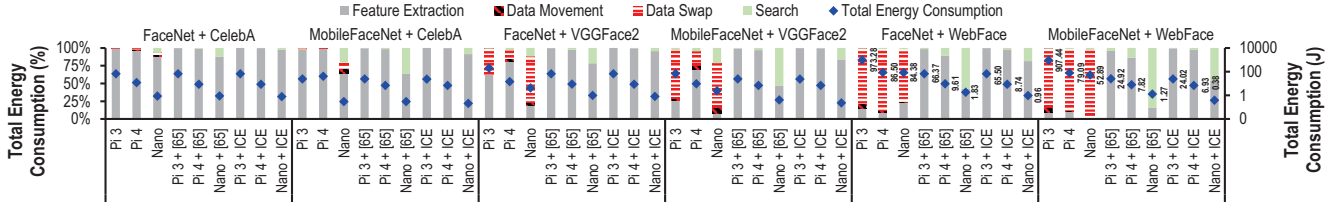


Figure 16: Comparison of the measured total energy of edge systems and simulated total energy of systems with [65] and ICE.

in [12], [15], [79] focus on volatile-based in-place logic computation, e.g., copy, AND, XOR, XNOR. However, the issue of how to implement multi-bit multiplication is ignored in these works. Moreover, these solutions need to consume power to register data in standby mode and store back data after wake-up. Most volatile solutions do not have sufficient capacity for weight storage because they need to allocate part of the capacity for temporary data storage of processors. ICE proposes a NAND-based nvIMC solution with high-density 3D NAND without data movement and energy consumption in the standby mode.

Nonvolatile Memory-based IMC: Works in [17], [28], [37], [47], [50], [65], [80], [92], [95], [96], [97] feature their nonvolatile storage ability of weights after power-off, and achieving VVM operations in the nonvolatile memory for minimizing data movement. Works in [43], [89] feature nonvolatile-based in-place logic computation, e.g., ADD, NAND, NOR; however, they do not mention how to implement multi-bit multiplication. Due to the immature manufacturing process, emerging memories, e.g., ReRAM, and PCM, still need to overcome the issues of resistance variation [37] which could lead to imprecise VVM.

VIII. CONCLUSION

This paper presents a low-energy and high-performance ICE with cognitive 3D NAND for vector similarity search (VSS) applications. Cognitive 3D NAND can support INT8/INT4 vector-vector multiplication (VVM) operations by exploiting digital nvIMC techniques. Specifically, cognitive 3D NAND shows the feasibility of leveraging the 16KB page buffer and two's complement digital accumulator to accomplish VVM operations without ADCs and DACs. Moreover, the bit-error-tolerance data encoding strengthens in mitigating the error-bit effect. According to our designs and estimation, it only

requires 1.6% area overhead in a commercial circuit-under-array 3D NAND chip to achieve INT8/INT4 VVM operations. With the support of cognitive 3D NAND chips, ICE can process data locally, instead of in processors, so as to suppress data movement and swap. Moreover, the system provides data flow optimization for searching to quickly output the most similar answer of a given input. Therefore, ICE can accelerate VSS applications by suppressing the high search energy and long search latency of massive data movement and dataset processing and offloading the computational burden. Compared to the von Neumann approach using some commercial edge systems (Pi3, Pi4, and Jetson Nano) with MobileFaceNet and the CASIA-WebFace dataset, the measured and simulated results show that the proposed ICE enhances the system execution time by $17\times$ to $95\times$ and energy efficiency by $11\times$ to $140\times$.

ACKNOWLEDGMENT

This work is supported by Macronix's internal R&D program. The authors would like to thank the Product Design and Engineering Center for chip design and discussion, the Technology Development Center for fabrication, and the Product Engineering Center for chip testing support. This work is also partially supported by the Macronix-NTU industrial collaboration project under grant no. 111HZA41001, Macronix-NTHU industrial collaboration project under grant no. 108A2073EA, Academia Sinica under grant nos. AS-IA-111-M01, AS-GCS-110-08, AS-CDA-107-M05, and National Science and Technology Council under grant nos. 111-2223-E-001-001 and 111-2221-E-001-013-MY3.

REFERENCES

- [1] Benchmarking Edge Computing. Accessed: 2022-08-20. [Online]. Available: <https://aallan.medium.com/benchmarking-edge-computing-ce3f13942245/>
- [2] Dell. Accessed: 2022-08-20. [Online]. Available: https://i.dell.com/sites/doccontent/shared-content/data-sheets/en/Documents/512e_4Kn_Disk_Formats_120413.pdf
- [3] eGroupAI. Accessed: 2022-08-20. [Online]. Available: <https://www.egroupai.com/face-iot-lite-sdk>
- [4] FaceMe. Accessed: 2022-08-20. [Online]. Available: <https://www.cyberlink.com/faceme/insights/articles/256/build-facial-recognition-arm-based-aiot-device>
- [5] Nvidia Jetson Nano. Accessed: 2022-08-20. [Online]. Available: <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/education-projects/>
- [6] Overview of SSD Structure and Basic Working Principle(2). Accessed: 2022-08-20. [Online]. Available: <https://www.elinfor.com/knowledge/overview-of-ssd-structure-and-basic-working-principle2-p-11204>
- [7] Power Consumption Benchmarks. Accessed: 2022-08-20. [Online]. Available: <https://www.pidramble.com/wiki/benchmarks/power-consumption/>
- [8] Rakinda. Accessed: 2022-08-20. [Online]. Available: <https://www.rakinda.com/en/productdetail/83/132/237.html>
- [9] Raspberry Pi 3. Accessed: 2022-08-20. [Online]. Available: <https://www.raspberrypi.org/products/raspberry-pi-3-model-b/>
- [10] Raspberry Pi 4. Accessed: 2022-08-20. [Online]. Available: <https://www.raspberrypi.org/products/raspberry-pi-4-model-b/>
- [11] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "TensorFlow: A System for Large-Scale Machine Learning," *USENIX Symposium on Operating Systems Design and Implementation*, 2016, pp. 265–283.
- [12] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, "Compute Caches," *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 481–492.
- [13] R. Agarwal and M. Marrow, "A Closed-Form Expression for Write Amplification in NAND Flash," *IEEE Globecom Workshops*, 2010, pp. 1846–1850.
- [14] P. Anand, A. K. Singh, S. Srivastava, and B. Lall, "Few Shot Speaker Recognition Using Deep Neural Networks," *arXiv*, 2019.
- [15] S. Angizi, N. A. Fahmi, W. Zhang, and D. Fan, "PIM-Assembler: A Processing-In-Memory Platform for Genome Assembly," *ACM/IEEE Design Automation Conference (DAC)*, 2020, pp. 1–6.
- [16] R. Banner, Y. Nahshan, E. Hoffer, and D. Soudry, "Post-Training 4-bit Quantization of Convolution Networks for Rapid-Deployment," *arXiv*, 2018.
- [17] M. Bavandpour, S. Sahay, M. R. Mahmoodi, and D. Strukov, "Efficient Mixed-Signal Neurocomputing via Successive Integration and Rescaling," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 3, pp. 823–827, 2020.
- [18] A. Biswas and A. P. Chandrakasan, "Conv-RAM: An Energy-Efficient SRAM with Embedded Convolution Computation for Low-Power CNN-Based Machine Learning Applications," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2018, pp. 488–490.
- [19] R. Bose and D. Ray-Chaudhuri, "On A Class of Error Correcting Binary Group Codes," *Information and Control*, vol. 3, no. 1, pp. 68–79, 1960.
- [20] H. Cai, C. Gan, L. Zhu, and S. Han, "TinyTL: Reduce Memory, Not Parameters for Efficient On-Device Learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11 285–11 297.
- [21] Q. Cao, L. Shen, W. Xie, O. M. Parkhi, and A. Zisserman, "VGGFace2: A Dataset for Recognising Faces Across Pose and Age," *IEEE International Conference on Automatic Face and Gesture Recognition*, 2018, pp. 67–74.
- [22] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep Clustering for Unsupervised Learning of Visual Features," *European Conference on Computer Vision (ECCV)*, 2018, pp. 132–149.
- [23] J. Chang, Y. Chen, G. Chan, H. Cheng, P. Wang, Y. Lin, H. Fujiwara, R. Lee, H. Liao, P. Wang, G. Yeap, and Q. Li, "A 5nm 135Mb SRAM in EUV and High-Mobility-Channel FinFET Technology with Metal Coupling and Charge-Sharing Write-Assist Circuitry Schemes for High-Density and Low-VMIN Applications," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 238–240.
- [24] Y.-H. Chang, J.-W. Hsieh, and T.-W. Kuo, "Endurance Enhancement of Flash-Memory Storage Systems: An Efficient Static Wear Leveling Design," *ACM/IEEE Design Automation Conference (DAC)*, 2007, pp. 212–217.
- [25] M. Chen, F. Herrera, and K. Hwang, "Cognitive Computing: Architecture, Technologies and Intelligent Applications," *IEEE Access*, vol. 6, pp. 19 774–19 783, 2018.
- [26] S. Chen, Y. Liu, X. Gao, and Z. Han, "MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices," *Chinese Conference on Biometric Recognition*, 2018, pp. 428–438.
- [27] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," *International Conference on Machine Learning*, 2020, pp. 1597–1607.

- [28] W. Chen, K. Li, W. Lin, K. Hsu, P. Li, C. Yang, C. Xue, E. Yang, Y. Chen, Y. Chang, T. Hsu, Y. King, C. Lin, R. Liu, C. Hsieh, K. Tang, and M. Chang, "A 65nm 1Mb Nonvolatile Computing-in-Memory ReRAM Macro with Sub-16ns Multiply-and-Accumulate for Binary DNN AI Edge Processors," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2018, pp. 494–496.
- [29] X. Chen and K. He, "Exploring Simple Siamese Representation Learning," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 750–15 758.
- [30] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cuDNN: Efficient Primitives for Deep Learning," *arXiv*, 2014.
- [31] A. Corporation, "HSPICE User Guide: Simulation and Analysis," 2002.
- [32] N. Dehak, R. Dehak, J. R. Glass, D. Reynolds, and P. Kenny, "Cosine Similarity Scoring without Score Normalization Techniques," *Odyssey*, 2010.
- [33] J. Do, V. C. Ferreira, H. Bobarshad, M. Torabzadehkashi, S. Rezaei, A. Heydarigorji, D. Souza, B. F. Goldstein, L. Santiago, M. S. Kim, P. M. V. Lima, F. M. G. França, and V. Alves, "Cost-Effective, Energy-Efficient, and Scalable Storage Computing for Large-Scale AI Applications," *ACM Transactions on Storage*, vol. 16, no. 4, pp. 1–37, 2020.
- [34] I. Dokmanic, R. Parhizkar, J. Ranieri, and M. Vetterli, "Euclidean Distance Matrices: Essential Theory, Algorithms, and Applications," *IEEE Signal Processing Magazine*, vol. 32, no. 6, pp. 12–30, 2015.
- [35] Q. Dong, M. E. Sinangil, B. Erbagci, D. Sun, W. Khwa, H. Liao, Y. Wang, and J. Chang, "A 351TOPS/W and 372.4GOPS Compute-in-Memory SRAM Macro in 7nm FinFET CMOS for Machine-Learning Applications," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 242–244.
- [36] Q. Fan, W. Zhuo, C.-K. Tang, and Y.-W. Tai, "Few-Shot Object Detection with Attention-RPN and Multi-Relation Detector," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 4013–4022.
- [37] A. Fantini, G. Gorine, R. Degraeve, L. Goux, C.-Y. Chen, A. Redolfi, S. Clima, A. Cabrini, G. Torelli, and M. Jurczak, "Intrinsic Program Instability in HfO₂ RRAM and Consequences on Program Algorithms," *IEEE International Electron Devices Meeting (IEDM)*, 2015, pp. 7–5.
- [38] H. Fujiwara, C. Y. Lin, H. Y. Pan, C. H. Lin, P. Y. Huang, K. C. Lin, J. J. Liaw, Y. H. Chen, H. J. Liao, and J. Chang, "A 7nm 2.1GHz Dual-Port SRAM with WL-RC Optimization and Dummy-Read-Recovery Circuitry to Mitigate Read-Disturb-Write Issue," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019, pp. 390–392.
- [39] R. Gallager, "Low-Density Parity-Check Codes," *IRE Transactions on Information Theory*, vol. 8, no. 1, pp. 21–28, 1962.
- [40] S. K. Gonugondla, M. Kang, and N. Shanbhag, "A 42pJ/Decision 3.12TOPS/W Robust In-Memory Machine Learning Classifier with On-Chip Training," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2018, pp. 490–492.
- [41] A. Gordo, J. Almazán, J. Revaud, and D. Larlus, "Deep Image Retrieval: Learning Global Representations for Image Search," *European Conference on Computer Vision (ECCV)*, 2016, pp. 241–257.
- [42] B. Gu, A. S. Yoon, D. H. Bae, I. Jo, J. Lee, J. Yoon, J. U. Kang, M. Kwon, C. Yoon, S. Cho, J. Jeong, and D. Chang, "Biscuit: A Framework for Near-Data Processing of Big Data Workloads," *IEEE/ACM International Symposium on Computer Architecture (ISCA)*, 2016, pp. 153–165.
- [43] S. Gupta, M. Imani, and T. Rosing, "Felix: Fast and Energy-Efficient Logic in Memory," *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2018, pp. 1–7.
- [44] M. Hatamian and G. Cash, "A 70-MHz 8-bit $\times$ 8-bit Parallel Pipelined Multiplier in 2.5 μ m CMOS," *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 21, no. 4, pp. 505–513, 1986.
- [45] T. Higuchi, T. Kodama, K. Kato, R. Fukuda, N. Tokiwa, M. Abe, T. Takagiwa, Y. Shimizu, J. Musha, K. Sakurai, J. Sato, T. Utsumi, K. Yoneya, Y. Suematsu, T. Hashimoto, T. Hioaka, K. Yanagidaira, M. Kojima, J. Matsuno, K. Shiraishi, K. Yamamoto, S. Hayashi, T. Hashiguchi, K. Inuzuka, A. Sugahara, M. Honma, K. Tsunoda, K. Yamamoto, T. Sugimoto, T. Fujimura, M. Kaneko, H. Date, O. Kobayashi, T. Minamoto, R. Tachibana, I. Yamaguchi, J. Lee, V. Ramachandra, S. Rajendra, T. Tang, S. Darne, J. Lee, J. Li, T. Miwa, R. Yamashita, H. Sugawara, N. Ookuma, M. Kano, H. Mizukoshi, Y. Kuniyoshi, M. Watanabe, K. Akiyama, H. Mori, A. Arimizu, Y. Katano, M. Ehama, H. Maejima, K. Hosono, and M. Yoshihara, "A 1Tb 3b/Cell 3D-Flash Memory in A 170+ Word-Line-Layer Technology," *IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 428–430.
- [46] A. Hocquenghem, "Codes Correcteurs d'erreurs," *Chiffers*, vol. 2, pp. 147–156, 1959.
- [47] H.-W. Hu, W.-C. Wang, C.-K. Chen, Y.-C. Lee, B.-R. Lin, H.-M. Wang, Y.-P. Lin, Y.-C. Lin, C.-C. Hsieh, C.-M. Hu, Y.-T. Lai, H.-S. Chen, Y.-H. Chang, H.-P. Li, T.-W. Kuo, K.-C. Wang, M.-F. Chang, C.-H. Hung, and C.-Y. Lu, "A 512Gb In-Memory-Computing 3D-NAND Flash Supporting Similar-Vector-Matching Operations on Edge-AI Devices," *IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, 2022, pp. 138–140.
- [48] H. Hu, Y. Wang, L. Yang, P. Komlev, L. Huang, X. Chen, J. Huang, Y. Wu, M. Merchant, and A. Sacheti, "Web-Scale Responsive Visual Search at Bing," 2018.
- [49] X. S. Hu, M. Niemier, A. Kazemi, A. F. Laguna, K. Ni, R. Rajaei, M. M. Sharifi, and X. Yin, "In-Memory Computing with Associative Memories: A Cross-Layer Perspective," *IEEE International Electron Devices Meeting (IEDM)*, 2021, pp. 25.2.1–25.2.4.

- [50] M. Imani, S. Pampana, S. Gupta, M. Zhou, Y. Kim, and T. Rosing, "DUAL: Acceleration of Clustering Algorithms using Digital-Based Processing In-Memory," *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020, pp. 356–371.
- [51] Y. Jing, D. Liu, D. Kislyuk, A. Zhai, J. Xu, J. Donahue, and S. Tavel, "Visual Search at Pinterest," *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2015, pp. 1889–1898.
- [52] I. Jo, D.-H. Bae, A. S. Yoon, J.-U. Kang, S. Cho, D. D. G. Lee, and J. Jeong, "YourSQL: A High-Performance Database System Leveraging In-Storage Computing," *Proceedings of the VLDB Endowment*, vol. 9, no. 12, pp. 924–935, 2016.
- [53] B. Kang, Z. Liu, X. Wang, F. Yu, J. Feng, and T. Darrell, "Few-Shot Object Detection via Feature Reweighting," *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 8420–8429.
- [54] W. S. Khwa, J. J. Chen, J. F. Li, X. Si, E. Y. Yang, X. Sun, R. Liu, P. Y. Chen, Q. Li, S. Yu, and M. F. Chang, "A 65nm 4Kb Algorithm-Dependent Computing-in-Memory SRAM Unit-Macro with 2.3ns and 55.8TOPS/W Fully Parallel Product-Sum Operation for Binary DNN Edge Processors," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2018, pp. 496–498.
- [55] S. Kim, J.-S. No, H. Chung, and D.-J. Shin, "Quasi-Cyclic Low-Density Parity-Check Codes With Girth Larger Than 12," *IEEE Transactions on Information Theory*, vol. 53, no. 8, pp. 2885–2891, 2007.
- [56] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated Optimization: Distributed Machine Learning for On-Device Intelligence," *arXiv*, 2016.
- [57] L. Wang, Y. Zhang, and J. Feng, "On the Euclidean Distance of Images," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1334–1339, 2005.
- [58] I. Lee, H. Jeong, S. Baek, S. Gupta, C. Park, D. Seo, J. Choi, J. Kim, H. Kim, J. Kang, S. Jang, D. Moon, S. Han, T. Kim, J. Lim, Y. Park, H. Hwang, J. Kang, J. Choi, and T. Song, "A Voltage and Temperature Tracking SRAM Assist Supporting 740mV Dual-Rail Offset for Low-Power and High-Performance Applications in 7nm EUV FinFET Technology," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019, pp. 392–394.
- [59] S.-T. Lee and J.-H. Lee, "Neuromorphic Computing Using NAND Flash Memory Architecture with Pulse Width Modulation Scheme," *Frontiers in Neuroscience*, vol. 14, pp. 945, 2020.
- [60] A. Li, T. Luo, T. Xiang, W. Huang, and L. Wang, "Few-Shot Learning with Global Class Representations," *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 9715–9724.
- [61] H. Li, W.-C. Chen, A. Levy, C.-H. Wang, H. Wang, P.-H. Chen, W. Wan, H.-S. P. Wong, and P. Raina, "One-Shot Learning with Memory-Augmented Neural Networks Using A 64-kbit, 118 GOPS/W RRAM-Based Non-Volatile Associative Memory," *Symposium on VLSI Technology*, 2021, pp. 1–2.
- [62] Lie Lu, Liu Wenyin, and Hong-Jiang Zhang, "Audio Textures: Theory and Applications," *IEEE Transactions on Speech and Audio Processing*, vol. 12, no. 2, pp. 156–167, 2004.
- [63] Y.-Y. Lin, F.-M. Lee, M.-H. Lee, W.-C. Chen, H.-L. Lung, K.-C. Wang, and C.-Y. Lu, "A Novel Voltage-Accumulation Vector-Matrix Multiplication Architecture Using Resistor-Shunted Floating Gate Flash Memory Device for Low-Power and High-Density Neural Network Applications," *IEEE International Electron Devices Meeting (IEDM)*, 2018, pp. 2–4.
- [64] Z. Liu, P. Luo, X. Wang, and X. Tang, "Large-scale CelebFaces Attributes (CelebA) Dataset," *Retrieved August*, vol. 15, no. 2018, pp. 11, 2018.
- [65] H.-T. Lue, P.-K. Hsu, M.-L. Wei, T.-H. Yeh, P.-Y. Du, W.-C. Chen, K.-C. Wang, and C.-Y. Lu, "Optimal Design Methods to Transform 3D NAND Flash into A High-Density, High-Bandwidth and Low-Power Nonvolatile Computing-in-Memory (nvCIM) Accelerator for Deep-Learning Neural Networks (DNN)," *IEEE International Electron Devices Meeting (IEDM)*, 2019, pp. 38–1.
- [66] V. S. Mailthody, Z. Qureshi, W. Liang, Z. Feng, S. G. De Gonzalo, Y. Li, H. Franke, J. Xiong, J. Huang, and W.-m. Hwu, "Deepstore: In-Storage Acceleration for Intelligent Queries," *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2019, pp. 224–238.
- [67] H. Nguyen and L. Bai, "Cosine Similarity Metric Learning for Face Verification," *Asian Conference on Computer Vision*, vol. 6493, 2010, pp. 709–720.
- [68] M. Norouzi, D. J. Fleet, and R. R. Salakhutdinov, "Hamming Distance Metric Learning," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012.
- [69] H. Oh Song, Y. Xiang, S. Jegelka, and S. Savarese, "Deep Metric Learning via Lifted Structured Feature Embedding," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4004–4012.
- [70] S. A. Osia, A. Shahin Shamsabadi, S. Sajadmanesh, A. Taheri, K. Katevas, H. R. Rabiee, N. D. Lane, and H. Haddadi, "A Hybrid Deep Learning Architecture for Privacy-Preserving Mobile Analytics," *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 4505–4518, 2020.
- [71] J. R. Parker and B. Behm, "Creating Audio Textures by Example: Tiling and Stitching," *IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 4, 2004.
- [72] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8026–8037.
- [73] S. Reed, Z. Akata, H. Lee, and B. Schiele, "Learning Deep Representations of Fine-Grained Visual Descriptions," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 49–58.

- [74] T. J. Richardson and R. L. Urbanke, "Efficient Encoding of Low-Density Parity-Check Codes," *IEEE Transactions on Information Theory*, vol. 47, no. 2, pp. 638–656, 2001.
- [75] T. J. Richardson and R. L. Urbanke, "The Capacity of Low-Density Parity-Check Codes under Message-Passing Decoding," *IEEE Transactions on Information Theory*, vol. 47, no. 2, pp. 599–618, 2001.
- [76] L. Saiz, J. Gracia, D. Gil, P. Gil-Vicente, and J.-C. Baraza-Calvo, "Reducing the Overhead of BCH Codes: New Double Error Correction Codes," *Electronics*, vol. 9, 2020.
- [77] C. Schmid and R. Mohr, "Local Grayvalue Invariants for Image Retrieval," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 19, no. 5, pp. 530–535, 1997.
- [78] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [79] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology," *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 273–287.
- [80] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars," *IEEE/ACM International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.
- [81] N. Shibata, K. Kanda, T. Shimizu, J. Nakai, O. Nagao, N. Kobayashi, M. Miakashi, Y. Nagadomi, T. Nakano, T. Kawabe, T. Shibuya, M. Sako, K. Yanagidaira, T. Hashimoto, H. Date, M. Sato, T. Nakagawa, H. Takamoto, J. Musha, T. Minamoto, M. Uda, D. Nakamura, K. Sakurai, T. Yamashita, J. Zhou, R. Tachibana, T. Takagiwa, T. Sugimoto, M. Ogawa, Y. Ochi, K. Kawaguchi, M. Kojima, T. Ogawa, T. Hashiguchi, R. Fukuda, M. Masuda, K. Kawakami, T. Someya, Y. Kajitani, Y. Matsumoto, N. Morozumi, J. Sato, N. Raghunathan, Y. L. Koh, S. Chen, J. Lee, H. Nasu, H. Sugawara, K. Hosono, T. Hisada, T. Kaneko, and H. Nakamura, "A 1.33Tb 4-bit/Cell 3D-Flash Memory on A 96-Word-Line-Layer Technology," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019, pp. 210–212.
- [82] X. Si, Y. Tu, W. Huang, J. Su, P. Lu, J. Wang, T. Liu, S. Wu, R. Liu, Y. Chou, Z. Zhang, S. Sie, W. Wei, Y. Lo, T. Wen, T. Hsu, Y. Chen, W. Shih, C. Lo, R. Liu, C. Hsieh, K. Tang, N. Lien, W. Shih, Y. He, Q. Li, and M. Chang, "A 28nm 64Kb 6T SRAM Computing-in-Memory Macro with 8b MAC Operation for AI Edge Chips," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 246–248.
- [83] X. Si, J.-J. Chen, Y.-N. Tu, W.-H. Huang, J.-H. Wang, Y.-C. Chiu, W.-C. Wei, S.-Y. Wu, X. Sun, R. Liu, S. Yu, R.-S. Liu, C.-C. Hsieh, K.-T. Tang, Q. Li, and M.-F. Chang, "A Twin-8T SRAM Computation-in-Memory Macro for Multiple-Bit CNN-Based Machine Learning," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019, pp. 396–398.
- [84] H. V. Simhadri, G. Williams, M. Aumüller, M. Douze, A. Babenko, D. Baranchuk, Q. Chen, L. Hosseini, R. Krishnaswamy, G. Srinivasa *et al.*, "Results of the NeurIPS'21 Challenge on Billion-Scale Approximate Nearest Neighbor Search," *arXiv*, 2022.
- [85] J. Su, X. Si, Y. Chou, T. Chang, W. Huang, Y. Tu, R. Liu, P. Lu, T. Liu, J. Wang, Z. Zhang, H. Jiang, S. Huang, C. Lo, R. Liu, C. Hsieh, K. Tang, S. Sheu, S. Li, H. Lee, S. Chang, S. Yu, and M. Chang, "A 28nm 64Kb Inference-Training Two-Way Transpose Multibit 6T SRAM Compute-in-Memory Macro for AI Edge Chips," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 240–242.
- [86] S.-Y. Chung, G. D. Forney, T. J. Richardson, and R. Urbanke, "On the Design of Low-Density Parity-Check Codes within 0.0045 dB of the Shannon Limit," *IEEE Communications Letters*, vol. 5, no. 2, pp. 58–60, 2001.
- [87] D. Sztahó, G. Szaszák, and A. Beke, "Deep Learning Methods in Speaker Recognition: A Review," *arXiv*, 2019.
- [88] K. Tieu and P. Viola, "Boosting Image Retrieval," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2000, pp. 228–235.
- [89] M. S. Truong, E. Chen, D. Su, L. Shen, A. Glass, L. R. Carley, J. A. Bain, and S. Ghose, "RACER: Bit-Pipelined Processing Using Resistive Memory," *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 100–116.
- [90] J. Wang, J. Zhang, W. Bao, X. Zhu, B. Cao, and P. S. Yu, "Not Just Privacy: Improving Performance of Private Deep Learning in Mobile Cloud," *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2018, pp. 2407–2416.
- [91] J. Wang, D. Park, Y. Papakonstantinou, and S. Swanson, "SSD In-Storage Computing for Search Engines," *IEEE Transactions on Computers*, pp. 1–1, 2016.
- [92] P. Wang, F. Xu, B. Wang, B. Gao, H. Wu, H. Qian, and S. Yu, "Three-Dimensional NAND Flash for Vector–Matrix Multiplication," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 4, pp. 988–991, 2019.
- [93] Y. Wang, Q. Yao, J. T. Kwok, and L. M. Ni, "Generalizing from A Few Examples: A Survey on Few-Shot Learning," *ACM Computing Surveys (CSUR)*, vol. 53, no. 3, pp. 1–34, 2020.
- [94] S. Xie, C. Ni, A. Sayal, P. Jain, F. Hamzaoglu, and J. P. Kulkarni, "eDRAM-CIM: Compute-in-Memory Design with Reconfigurable Embedded-Dynamic-Memory Array Realizing Adaptive Data Converters and Charge-Domain Computing," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021, pp. 248–250.
- [95] C. Xue, T. Huang, J. Liu, T. Chang, H. Kao, J. Wang, T. Liu, S. Wei, S. Huang, W. Wei, Y. Chen, T. Hsu, Y. Chen, Y. Lo, T. Wen, C. Lo, R. Liu, C. Hsieh, K. Tang, and M. Chang, "A 22nm 2Mb ReRAM Compute-in-Memory Macro with 121-28TOPS/W for Multibit MAC Computing for Tiny AI Edge Devices," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2020, pp. 244–246.

- [96] C.-X. Xue, J.-M. Hung, H.-Y. Kao, Y.-H. Huang, S.-P. Huang, F.-C. Chang, P. Chen, T.-W. Liu, C.-J. Jhang, C.-I. Su, W.-S. Khwa, C.-C. Lo, R.-S. Liu, C.-C. Hsieh, K.-T. Tang, Y.-D. Chih, T.-Y. J. Chang, and M.-F. Chang, "A 22nm 4Mb 8b-Precision ReRAM Computing-in-Memory Macro with 11.91 to 195.7TOPS/W for Tiny AI Edge Devices," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021, pp. 245–247.
- [97] C. Xue, W. Chen, J. Liu, J. Li, W. Lin, W. Lin, J. Wang, W. Wei, T. Chang, T. Chang, T. Huang, H. Kao, S. Wei, Y. Chiu, C. Lee, C. Lo, Y. King, C. Lin, R. Liu, C. Hsieh, K. Tang, and M. Chang, "A 1Mb Multibit ReRAM Computing-in-Memory Macro with 14.6ns Parallel MAC Computing Time for CNN Based AI Edge Processors," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019, pp. 388–390.
- [98] F. Yang, A. Kale, Y. Bubnov, L. Stein, Q. Wang, H. Kiapour, and R. Piramuthu, "Visual Search at eBay," *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2017, pp. 2101–2110.
- [99] J. Yang, Y. Kong, Z. Wang, Y. Liu, B. Wang, S. Yin, and L. Shi, "Sandwich-RAM: An Energy-Efficient In-Memory BWN Architecture with Pulse-Width Modulation," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2019, pp. 394–396.
- [100] D. Yi, Z. Lei, S. Liao, and S. Z. Li, "Learning Face Representation from Scratch," *arXiv*, 2014.
- [101] L. Zhang, Y. Zhang, J. Tang, K. Lu, and Q. Tian, "Binary Code Ranking with Weighted Hamming Distance," *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013, pp. 1586–1593.
- [102] Y. Zhang, P. Pan, Y. Zheng, K. Zhao, Y. Zhang, X. Ren, and R. Jin, "Visual Search at Alibaba," *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2018, pp. 993–1001.
- [103] Y. Zhou, F. Wu, P. Huang, X. He, C. Xie, and J. Zhou, "An Efficient Page-Level FTL to Optimize Address Translation in Flash Memory," *European Conference on Computer Systems*, 2015, pp. 1–16.